04_info_2610161312

Cours d'informatique

Jean Decroocq

CPGE TSI – 2025–2027
Lycée polyvalent de Cachan

Dernière mise à jour : 16 octobre 2026
Communiquer une anomalie : jeandecroocq [at] hotmail.com

Table des matières

I Méthodes de programmation **5**

1 Fondements **6**
1.1 Variables et types . . . 6
1.2 Fonctions . . . 6
1.3 Algorithmique et programmation . . . 7
1.4 Modules . . . 8

2 Méthodes de programmation **9**
2.1 Spécification d'une fonction . . . 9
2.2 Assertion . . . 9
2.3 Génération de test . . . 10

3 Terminaison et correction des algorithmes **11**
3.1 Enjeux . . . 11
3.2 Variant de boucle . . . 11
3.3 Invariant de boucle . . . 11
3.4 Exemple . . . 11

4 Listes et chaînes de caractères **13**
4.1 Tableaux et listes . . . 13
4.2 Chaînes de caractères . . . 14

5 Dictionnaires **15**
5.1 Création et modification . . . 15
5.2 Commandes principales . . . 15

6 Lecture d'un fichier texte de données **17**
6.1 Encodage et format . . . 17
6.2 Lecture d'un fichier texte . . . 17
6.3 Lecture via Numpy . . . 18

7 Numpy **19**
7.1 Notion . . . 19
7.2 Tableaux numpy . . . 19
7.3 Opérations sur les tableaux numpy . . . 22

8 Matplotlib **23**
8.1 Tracé de courbes . . . 23

9 Récursivité **25**
9.1 Principe . . . 25
9.2 Analyse des fonctions récursives . . . 25

10 Matrices de pixels et images **26**
10.1 Formats de représentation d'image . . . 26
10.2 Manipulation des matrices de pixels . . . 26
10.3 Transformation d'images . . . 26
10.4 Modification par convolution : filtrage . . . 27
10.5 Détection de contours . . . 28

11 Algorithmes gloutons et dichotomiques **29**
11.1 Algorithmes gloutons . . . 29
11.2 Algorithmes dichotomiques . . . 29

12 Algorithmes de tri **31**
12.1 Introduction . . . 31
12.2 Tri par sélection . . . 31
12.3 Tri par insertion . . . 32
12.4 Tri par fusion . . . 32
12.5 Tri par comptage . . . 33
12.6 Tri rapide . . . 33
12.7 Tri à bulle . . . 35

13 Complexité algorithmique **36**
13.1 Performances et vocabulaire . . . 36
13.2 Évaluation avec des boucles itératives . . . 36
13.3 Évaluation avec `while` et récursivité . . . 37
13.4 Complexité des opérations natives Python . . . 37

14 Piles et files **39**
14.1 Piles (LIFO) . . . 39
14.2 Files (FIFO) . . . 40

15 Compléments **42**
15.1 Compléments en vrac :) . . . 42
15.2 Fonctions et méthodes (HP) . . . 42

II Représentation des nombres **44**

16 Représentation des nombres en mémoire **45**
16.1 Bases de numération . . . 45
16.2 Représentation des entiers . . . 45
16.3 Représentation des nombres réels . . . 46
16.4 Représentation des chaînes de caractères . . . 47

III Graphes **48**

17 Représentation native des graphes **49**
17.1 Liste de listes (liste d'adjacence) . . . 49
17.2 Dictionnaire d'adjacence . . . 49
17.3 Matrice d'adjacence . . . 50
17.4 Algorithmes de conversion et d'exploration . . . 51

18 Parcours de graphes **53**
18.1 Notions . . . 53
18.2 Parcours en largeur (BFS) . . . 53
18.3 Parcours en profondeur (DFS) . . . 55

IV Bases de données (SQL) **58**

19 Bases de données **59**
19.1 Architecture . . . 59
19.2 Vocabulaire et modélisation . . . 59
19.3 Extraction et restriction . . . 59
19.4 Tris, limites et opérations ensemblistes . . . 60
19.5 Croisement de données et jointures . . . 60
19.6 Agrégation et calculs . . . 61

V Algorithmes pour l'intelligence artificielle **62**

VI Algorithmique numérique **63**

20 Intégration numérique **64**
20.1 Introduction . . . 64
20.2 Méthode du point milieu . . . 64
20.3 Méthode des trapèzes . . . 65
20.4 Méthode de Simpson . . . 66

Première partie

Méthodes de programmation

Chapitre 1

Fondements

1.1 Variables et types

– Une variable constitue une référence vers un objet stocké en mémoire. L'affectation, réalisée via le signe `=`, lie un identifiant à une valeur. Le typage est dynamique : la nature de la variable est déterminée par la valeur qu'elle reçoit au moment de l'exécution, ce qui peut se vérifier avec `type(x)` qui renvoie le type de `x`. Enfin `print(x)` permet d'afficher la valeur de `x` dans la console.

– Les nombres entiers (`int`) supportent les opérations classiques `+`, `-`, `*`, `/` ainsi que la division euclidienne `//`, le modulo `%` et l'exponentiation `**`.

– Les nombres flottants (`float`) représentent les réels approximativement pour certaines valeurs décimales. Ils peuvent s'écrire en notation scientifique : `1.5e4` pour $1,5 \times 10^4$.

– Les booléens (`bool`) ne peuvent prendre que deux états : `True` ou `False`. Ils sont le résultat d'opérations de comparaison (`==`, `!=`, `<`, `>=`) et se combinent à l'aide des connecteurs logiques `not`, `and` et `or`.

– Les chaînes de caractères (`str`) sont des séquences immuables. On ne peut modifier un caractère directement (`s[0] = 'a'` lève une erreur). La concaténation s'effectue avec `+` et la répétition avec `*`.

– Les listes (`list`) sont des séquences mutables, ordonnées et hétérogènes, définies entre crochets. L'accès aux éléments se fait par un indice entier.

```
1 L = [10, 20, 30]
2 L[0] = 5          # modification : L vaut [5, 20, 30]
3 dernier = L[-1]   # accede au dernier element (30)
```

– Il est possible de convertir une valeur d'un type vers un autre (casting).
- `int(x)` convertit en entier.
  Un flottant et tronqué : `int(3.9)` donne `3`, `int(-2.7)` donne `-2`.
  Une chaîne doit représenter un entier : `int("42")` donne `42`, sinon une `ValueError` est levée : `int("3.5")` échoue.
- `float(x)` convertit en flottant : `float("1e-4")` donne `0.0001`.
- `str(x)` transforme n'importe quel objet en sa représentation textuelle, ce qui est indispensable pour l'affichage ou la concaténation.
- `bool(x)` convertit en booléen. En Python, la plupart des valeurs sont considérées comme vraies (`True`), sauf les valeurs "vides" ou nulles qui valent `False` : le nombre `0`, `0.0`, la chaîne vide `""`, les listes vides `[]`, et la valeur `None`.

1.2 Fonctions

– Une fonction encapsule un bloc d'instructions afin de le rendre réutilisable. Elle est définie par le mot-clé `def`, suivi de son nom et de ses paramètres entre parenthèses. Le bloc d'instructions

constituant le corps de la fonction doit impérativement être indenté.

– L'instruction `return` interrompt l'exécution de la fonction et renvoie la valeur spécifiée à l'endroit où la fonction a été appelée. Une fonction sans instruction `return` (ou avec un `return` vide) renvoie implicitement la valeur `None`.

```
def f(x, a):
    y = x ** 2 + a
    return y
```

– Les variables définies à l'intérieur d'une fonction ont une portée locale : elles n'existent que durant l'exécution de celle-ci et sont détruites ensuite. Elles ne peuvent pas modifier directement une variable globale immuable, sauf usage explicite (et déconseillé) du mot-clé `global`.

1.3 Algorithmique et programmation

– L'instruction conditionnelle dirige le flot d'exécution. Si la condition du `if` est fausse, le programme teste les éventuels `elif` successifs. Si aucune condition n'est vérifiée, le bloc `else` est exécuté.

```
if x > 0:
    signe = 1
elif x < 0:
    signe = -1
else:
    signe = 0
```

– La boucle bornée `for` est utilisée pour parcourir un itérable (liste, chaîne, tuple) ou réaliser un nombre d'itérations déterminé. On l'associe souvent au générateur `range(start, stop, step)`.
- `range(5)` génère `[0, 1, 2, 3, 4]` ;
- `range(1, 5)` génère `[1, 2, 3, 4]` ;
- `range(0, 10, 2)` génère `[0, 2, 4, 6, 8]`.

```
# moyenne d'une liste
L = [8, 2026, 18, 190]
s = 0
for val in L:
    s += val
m = s/len(L)
```

```
# somme des termes d'une suite géometrique
u0 = 1
q = 3
s = 0
for n in range(5):
    u = u0 * q**n # formule explicite, utilise n
    s += u # de u0 à u4

# variante pour illustrer les boucles
u = 1
s = 0
for n in range(5):
    s += u # de u0 à u4
    u = u * q # formule recursive, sans n
# u5 calculé mais non sommé
```

– La boucle non bornée `while` répète un bloc d'instructions tant qu'une condition booléenne reste vraie.

```
n = 0
while n < 10:
    print(n)
    n = n + 1  # indispensable pour que la condition devienne fausse
```

En Python, une collection (comme une liste) est évaluée à `False` si elle est vide, et `True` sinon. Ainsi, `while L:` est un raccourci élégant pour écrire `while len(L) > 0:` (tant que la liste n'est pas vide).

1.4 Modules

- Un module est une bibliothèque contenant un ensemble de fonctions et de variables pré-définies, permettant d'étendre les fonctionnalités natives de Python. Des modules standards comme `math`, `random` ou `time` sont disponibles immédiatement, tandis que d'autres comme `numpy` ou `matplotlib` doivent être installés.
- L'importation d'un module peut se faire de plusieurs manières. La méthode recommandée est l'import global, éventuellement avec un alias pour alléger l'écriture.

```
1 import math
2 y = math.cos(math.pi)
3
4 import numpy as np       # alias
5 T = np.array([1, 2, 3])
```

- Il est possible d'importer spécifiquement certaines fonctions dans l'espace de nommage courant avec `from ... import ...`. En revanche, l'importation totale via `from module import *` est fortement déconseillée car elle pollue l'espace de noms et peut créer des conflits (aliasing) difficiles à déboguer.
- La fonction `help(module)` ou `help(fonction)` permet d'accéder à la documentation intégrée, décrivant les paramètres attendus et le type de retour.

Chapitre 2

Méthodes de programmation

2.1 Spécification d'une fonction

- La lisibilité et la maintenabilité d'un code reposent sur une spécification claire des fonctions. Cela passe d'abord par le choix de noms explicites pour les variables et les fonctions, évitant les dénominations génériques (type `x`, `fct`, `res`).
- La *signature* d'une fonction définit ses entrées et ses sorties. Depuis Python 3.5, il est possible d'utiliser des annotations de type pour indiquer les types attendus des arguments et de la valeur de retour. Ces annotations sont purement indicatives et n'empêchent pas l'exécution en cas de non-respect.
- La documentation fonctionnelle s'insère via une chaîne de caractères spécifique, nommée *docstring*, placée entre triples guillemets juste après la signature. Elle décrit le rôle de la fonction, ses paramètres et ce qu'elle retourne. Elle est accessible via la commande `help(fonction)`.

```
def distance_euclidienne(p1: list, p2: list) -> float: # annot. de type
    """
    Calcule la distance euclidienne entre deux points.
    Entrées : p1 et p2 (list) contiennent les coordonnees (floats).
    Sortie  : distance (float).
    """ # fin documentation fonctionnelle

    d_carre = 0
    for i in range(len(p1)):
        d_carre += (p1[i] - p2[i])**2
    return d_carre**0.5
```

2.2 Assertion

- La validation des entrées permet de s'assurer que les arguments fournis respectent les préconditions nécessaires au bon fonctionnement de l'algorithme (types cohérents, dimensions compatibles, domaines de définition).
- L'instruction `assert condition, "message"` évalue une expression booléenne. Si celle-ci est fausse, le programme s'interrompt immédiatement en levant une `AssertionError` et affiche le message associé. Cela permet de détecter les bugs au plus tôt lors du développement.

```
def division(a, b):
    # precondition : le denominateur ne doit pas etre nul
    assert b != 0, "Division par zero impossible"
    return a / b

# exemple avec la fonction distance précédente
def distance_euclidienne(p1, p2):
    assert type(p1) == list and type(p2) == list, "Arguments invalides"
    assert len(p1) == len(p2), "Points de dimensions differentes"
    # ... suite du calcul ...
```

2.3 Génération de test

– Pour valider empiriquement qu'une fonction produit le résultat attendu, on rédige des tests unitaires. Un test consiste à appeler la fonction sur un cas connu et à comparer le résultat obtenu avec le résultat théorique attendu.

– Pour les nombres flottants, l'égalité stricte `==` est à proscrire en raison des erreurs d'arrondi inhérentes à la représentation binaire. On vérifie plutôt si l'écart entre la valeur calculée et la valeur attendue est inférieur à une précision arbitraire.

```python
def test_distance():
    # cas nominal : triplet Pythagoricien 3, 4, 5
    assert abs(distance_euclidienne([0, 0], [3, 4]) - 5.0) < 1e-9, "Echec test
        nominal"

    # cas limite : points confondus
    assert abs(distance_euclidienne([1, 2], [1, 2])) < 1e-9, "Echec test nul"

    print("Tests validés !")

test_distance()
```

Chapitre 3

Terminaison et correction des algorithmes

3.1 Enjeux

– En informatique, tester un programme sur quelques exemples ne suffit pas à garantir qu'il fonctionnera toujours correctement. Pour des algorithmes critiques, on a besoin de certitudes mathématiques sur deux aspects :
  • La *terminaison* : est-on sûr que la boucle ne va pas tourner à l'infini ?
  • La *correction* : est-on sûr que le résultat final est bien celui attendu ?

– Ces questions ne se posent que pour les boucles `while` et les fonctions récursives. Les boucles `for`, elles, s'arrêtent toujours car le nombre d'itérations est fixé.

3.2 Variant de boucle

– Pour prouver qu'une boucle s'arrête, on utilise un *variant*. L'idée est d'identifier une quantité numérique qui fonctionne comme un compte à rebours.

– Un variant de boucle est une expression entière qui doit respecter deux propriétés :
  • Être à valeurs dans les entiers naturels (positive ou nulle).
  • Décroître strictement à chaque passage dans la boucle.

– Le raisonnement est le suivant : comme on ne peut pas descendre indéfiniment en dessous de zéro avec des entiers, la boucle est mathématiquement obligée de s'arrêter.

– Exemple : Pour une boucle `while k < n` où `k` augmente de 1 à chaque tour, le variant classique est $n - k$. Cette quantité est positive (tant que la condition est vraie) et diminue de 1 à chaque tour.

3.3 Invariant de boucle

– Pour prouver qu'un algorithme calcule bien ce qu'on veut, on utilise un *invariant*. C'est une propriété logique qui reste vraie tout au long de l'exécution.

– La méthode fonctionne exactement comme une démonstration par récurrence en mathématiques :
  • Initialisation : On vérifie que la propriété est vraie juste avant d'entrer dans la boucle.
  • Hérédité : On montre que si la propriété est vraie au début d'un tour et qu'on exécute le corps de la boucle, elle reste vraie à la fin du tour.
  • Conclusion : À la sortie de la boucle, on combine l'invariant (qui est toujours vrai) et la condition d'arrêt (qui est devenue fausse) pour prouver que le résultat final est correct.

3.4 Exemple

– On considère la fonction suivante qui calcule la somme des éléments d'une liste L.

```
1 def somme(L):
```

```
    s = 0
    k = 0
    while k < len(L):
        s = s + L[k]
        k = k + 1
    return s
```

– Preuve de terminaison : On pose le variant $V = \text{len}(L) - k$. Au départ, V est un entier positif. À chaque tour, k augmente de 1, donc V diminue strictement de 1. La boucle termine.

– Preuve de correction : On choisit l'invariant : « la variable s contient la somme des k premiers éléments de la liste ».
  - Avant la boucle : $k = 0$ et $s = 0$. La somme de 0 élément vaut bien 0. L'invariant est vrai.
  - Pendant la boucle : On suppose que s est la somme des k premiers éléments. On ajoute $L[k]$ à s, puis on passe à $k + 1$. La variable s est maintenant la somme des $k + 1$ éléments. L'invariant est conservé.
  - À la fin : La boucle s'arrête quand $k = \text{len}(L)$. Comme l'invariant est toujours vrai, s contient donc la somme des $\text{len}(L)$ éléments, c'est-à-dire la somme totale. L'algorithme est correct.

Chapitre 4

Listes et chaînes de caractères

4.1 Tableaux et listes

– Les *listes* (`list`) sont des structures de données mutables, ordonnées et hétérogènes. Bien qu'appelées ainsi, elles sont implémentées sous forme de tableaux dynamiques contenant des références vers des objets.

– La construction d'une liste peut se faire par extension ou, de manière plus élégante et performante, par compréhension.

```
L1 = [0, 1, 4, 9, 16] # par extension
L2 = [k**2 for k in range(5)] # par comprehension (identique a L1)
L3 = [x for x in L1 if x % 2 == 0] # avec filtrage (nombres pairs)
```

– Lorsqu'une liste contient une autre liste, on accède à ses éléments ainsi : `L[a][b]`.

– Le mécanisme de tranchage (*slicing*) permet d'extraire une sous-partie de la liste en créant une nouvelle liste (copie). La syntaxe générale est `L[debut:fin:pas]`.
- `L[i:j]` : sélectionne les éléments de l'indice i inclus à j exclu.
- `L[i:]` : sélectionne de l'indice i jusqu'à la fin.
- `L[:j]` : sélectionne du début jusqu'à l'indice j exclu.
- `L[::-1]` : crée une copie renversée de la liste.

– Les listes étant mutables, elles disposent de méthodes agissant *in-place* (modifiant l'objet sans le renvoyer). Les plus courantes sont :
- `L.append(x)` : ajoute l'élément x à la fin de la liste.
- `L.pop()` : supprime et renvoie le dernier élément. `L.pop(i)` supprime l'élément à l'indice i.
- `L.sort()` : trie la liste en place.
- `L.reverse()` : inverse l'ordre des éléments en place.

– Voici quelques opérations usuelles :
- `L1 + L2`, `L * n` : concaténation, répétition.
- `len(L)` : longueur/nombre d'éléments.
- `max(L)`, `min(L)` : maximum, minimum.
- `sum(L)` : somme des éléments.
- `x in L` : test d'appartenance.

– Un point de vigilance majeur concerne l'aliasing. L'instruction `L2 = L1` ne copie pas la liste, mais crée une seconde référence vers le même objet en mémoire :

```
L1 = [1, 2, 3]
L2 = L1
L2[0] = 99
print(L1)  # affiche [99, 2, 3], L1 est modifiée aussi !
```

Pour obtenir une copie indépendante, il faut utiliser le tranchage complet `L[:]` ou la méthode `L.copy()` :

```python
L1 = [1, 2, 3]
L2 = L1[:]          # ou L2 = L1.copy()
L2[0] = 99
print(L1)  # affiche [1, 2, 3], L1 reste inchangée
```

4.2 Chaînes de caractères

– Les *chaînes de caractères* (`str`) sont des séquences ordonnées de caractères Unicode. Contrairement aux listes, elles sont immuables : il est impossible de modifier un caractère par affectation directe (`ch[0] = 'a'` lève une erreur `TypeError`).

– On définit une chaîne de caractère en l'entourant de guillemets simples, doubles, ou trois guillmets simples ou doubles. L'utilisation de guillemets simple permet d'utiliser des guillemets doubles dans la chaîne et vice-versa.

– La syntaxe de tranchage (*slicing*) et les fonctions universelles de séquences (`len()`, `in`) fonctionnent exactement comme pour les listes. La conversion d'un objet en chaîne se fait via le constructeur `str()`.

– Le langage propose des méthodes spécifiques pour la manipulation de texte :
- `s.find(motif)` : renvoie l'indice de la première occurrence du motif (ou -1 si absent).
- `s.count(motif)` : compte le nombre d'occurrences du motif.
- `s.strip()` : retire les espaces (et caractères invisibles) en début et fin de chaîne.
- `s.replace(old, new)` : remplace toutes les occurrences de la sous-chaîne `old` par `new`.

– Deux méthodes sont essentielles pour passer du type `str` au type `list` et inversement :
- `sep.join(liste)` : concatène les éléments d'une liste de chaînes en les séparant par la chaîne `sep`.
- `s.split(sep)` : découpe la chaîne `s` en une liste de sous-chaînes, en utilisant `sep` comme délimiteur.

```python
phrase = "Sciences du Numerique"
mots = phrase.split(" ")  # ['Sciences', 'du', 'Numerique']
reconst = "-".join(mots)  # "Sciences-du-Numerique"
```

Chapitre 5

Dictionnaires

5.1 Création et modification

- Le type `dict`, associé au *dictionnaire* ou *table d'association*, permet de stocker des couples *clé-valeur*. En effet, contrairement aux listes indexées par des entiers, les dictionnaires sont indexés par leurs *clés*, associées à leurs *valeurs*.
- Les clés doivent être de type hachable (immuable), comme les `int`, `float`, `str` ou `tuple`. Les valeurs, en revanche, peuvent être de n'importe quel type et mutables.
- L'intérêt majeur de cette structure est l'efficacité : l'accès à une valeur via sa clé se fait en temps constant moyen, quelle que soit la taille du dictionnaire.
- La création s'effectue via des accolades `{}` ou le constructeur `dict()`.

```
d1 = {} #dictionnaire vide
d2 = {'argent': 'silver', 'or': 'gold'} # par extension
d3 = {x: x**2 for x in range(5)} # par comprehension
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
d4 = dict(pomme=2, poire=5)
# {'pomme': 2, 'poire': 5}
```

- L'ajout ou la modification d'une entrée utilise la syntaxe d'affectation `d[cle] = valeur`. Si la clé existe déjà, l'ancienne valeur est écrasée (unicité des clés). Si elle n'existe pas, une nouvelle entrée est créée.
- De manière générale, prenons garde au type de la clé :

```
a = 3
d = {'a' : 0, a : 1}
print(d)        # {'a' : 0, 3 : 1}
print(d['a'])   # 0
print(d[3])     # 1
print(d[a])     # 1
```

- L'affectation `d1 = d2` ne crée pas de copie mais une nouvelle référence vers le même dictionnaire. Pour créer une copie indépendante, on emploie `d1 = d2.copy()` ou `d1 = dict(d2)`.

5.2 Commandes principales

- La fonction `len(d)` renvoie le nombre de paires clé-valeur stockées.
- Le test d'appartenance `k in d` (booléen) vérifie si la clé `k` est présente dans le dictionnaire. Cela ne teste pas la présence des valeurs.
- Pour parcourir un dictionnaire,
  - `d.keys()` itère sur les clés (comportement par défaut d'une boucle `for k in d`) ;
  - `d.values()` itère sur les valeurs ;

- `d.items()` itère sur les couples.

```
d = {'pomme': 2, 'poire': 5}
for k, v in d.items():
    print(k, "->", v) # affiche "pomme -> 2" etc.
```

– Pour récupérer une valeur,
  - `d[k]` renvoie la valeur associée à `k`, mais lève une erreur `KeyError` si la clé est absente ;
  - `d.get(k)` renvoie la valeur si elle existe, ou `None` sinon, sans provoquer d'erreur.

– La suppression d'un élément se fait via
  - la méthode `d.pop(k)`, qui supprime la clé `k` et renvoie la valeur associé ;
  - l'instruction `del d[k]`, qui supprime la clé `k` sans renvoyer de valeur.

– Conversion vers `dict` :

```
# a partir d'une liste de couples
liste = [('a',1), ('b',2)]
d = dict(liste)

# a partir de deux listes :
cles = ['a','b']
valeurs = [1,2]
d = dict(zip(cles, valeurs))
```

– Conversion depuis `dict` :

```
# liste des clés
list(d)              # clés
list(d.keys())

# liste des valeurs
list(d.values())

# listes des couples
list(d.items())
```

– On peut fusionner deux dictionnaires avec `d1.update(d2)`, qui ajoute les paires de `d2` à `d1` (écrasant les clés communes). Depuis Python 3.9, on peut aussi utiliser l'opérateur de fusion `d3 = d1 | d2`.

Chapitre 6

Lecture d'un fichier texte de données

6.1 Encodage et format

- Les fichiers textes utilisent des normes de codage binaire pour représenter les caractères. Si l'ASCII (7 bits) est universel, il ne gère pas les accents. La norme actuelle est l'Unicode (`utf-8`). Cependant, certains systèmes d'exploitation (Windows) utilisent par défaut des encodages historiques (comme `cp1252`).
- Pour garantir la portabilité du code et la bonne lecture des caractères accentués, il est impératif de toujours préciser l'argument `encoding="utf-8"` lors de l'ouverture d'un fichier.
- Le format CSV (*Comma Separated Values*) est un fichier texte simple où les données sont organisées par lignes. Au sein d'une ligne, les champs sont séparés par un caractère délimiteur.
- Bien que très répandu pour l'échange de données (tableurs, microcontrôleurs), ce format souffre d'une faible normalisation (gestion des séparateurs décimaux, des guillemets, etc.).

6.2 Lecture d'un fichier texte

- L'ouverture se fait via la fonction `open()`. L'utilisation du bloc contextuel `with` est préconisée car elle assure la fermeture automatique du fichier (`close()`) après usage. Mais on aurait pu écrire :

```
fichier = open("render.csv", mode='r', encoding="utf-8")
# ... traitement du fichier ...
fichier.close() # fermeture explicite
```

- La méthode `readlines()` renvoie une liste contenant toutes les lignes du fichier sous forme de chaînes de caractères.
- Le traitement des chaînes nécessite l'usage de caractères d'échappement (ex. `\n` pour le saut de ligne, `\t` pour la tabulation) et de deux méthodes clés :
  - `strip()` : supprime les caractères invisibles (espaces, sauts de ligne) en début et fin de chaîne ;
  - `split(separateur)` : découpe la chaîne en une liste de sous-chaînes.

```
temps, vitesse = [], []
with open("render.csv", mode='r', encoding="utf-8") as fichier:
    lignes = fichier.readlines()
    # on ignore la premiere ligne (en-tete)
    for ligne in lignes[1:]:
        # nettoyage et decoupage selon le point-virgule
        donnees = ligne.strip().split(';')
        # stockage et cast
        temps.append(float(donnees[0]))
        vitesse.append(float(donnees[1]))
```

- Le module `csv` facilite le découpage en gérant automatiquement les séparateurs. L'objet `csv.reader` permet d'itérer directement sur les lignes déjà découpées.

```python
import csv
temps, vitesse = [], []
with open("render.csv", mode="r", encoding="utf-8") as fichier:
    lecteur = csv.reader(fichier, delimiter=';')
    next(lecteur)  # saute la première ligne (en-tête)
    for ligne in lecteur:
        temps.append(float(ligne[0]))
        vitesse.append(float(ligne[1]))
```

6.3 Lecture via Numpy

– La fonction `np.loadtxt` permet de charger l'intégralité du fichier directement dans un tableau Numpy, en gérant la conversion de type et l'exclusion des en-têtes.

```python
import numpy as np

# chargement direct dans une matrice
data = np.loadtxt(
    "render.csv",
    delimiter=";",
    skiprows=1,         # ignore la ligne d'en-tete
    encoding="utf-8"
)

# extraction des colonnes
temps = data[:, 0]   # toutes les lignes, colonne 0
vitesse = data[:, 1] # toutes les lignes, colonne 1
```

Chapitre 7

Numpy

7.1 Notion

- Le module `numpy` est essentiel pour le calcul scientifique nécessitant la manipulation de grandes quantités de données. Il repose sur trois principes d'efficacité : le stockage sous forme de tableaux `ndarray`, la limitation des copies mémoire, et l'utilisation de fonctions vectorialisées pour éviter les boucles.
- Ces optimisations imposent des contraintes : les tableaux sont constitués d'éléments de même type (homogènes, souvent `np.float` ou `np.int64`) et leur taille est fixée à la création.
- Le chargement du module se fait traditionnellement via l'instruction : `import numpy as np`.

7.2 Tableaux numpy

- La création basique se fait via `np.array()` à partir d'une liste. L'attribut `.dtype` indique le type commun (ex. `int64`), différent des listes Python.

```
import numpy as np

a = np.array([1, 2, 3])
print(a)
# [1 2 3]

b = np.array([[1, 2, 3],
              [2, 3, 4],
              [0, 1, 0]])
print(b)
# [[1 2 3]
# [2 3 4]
# [0 1 0]]

print(type(a))
# <class 'numpy.ndarray'>
print(a.dtype)
# int64
```

- L'accès et la modification d'un élément spécifique s'effectuent en renseignant ses coordonnées entre crochets. Pour un tableau de dimension n, il est nécessaire de fournir n indices séparés par des virgules. Cette notation unifiée `a[i, j, ...]` est propre à Numpy et remplace l'enchaînement de crochets `a[i][j][...]` typique des listes natives imbriquées.

```
a = np.array([[1, 2],
              [3, 4]])

print(a[0, 1])
# 2

```

```
a[0, 0] = 7
print(a)
# [[7 2]
#  [3 4]]
```

– Pour les vecteurs (1D), on utilise `np.arange(start, stop, step)` qui accepte des pas flottants, ou `np.linspace(start, stop, num)` pour obtenir `num` points équitablement répartis (bornes incluses).

```
a = np.arange(0, 10, 2)
print(a)
# [0 2 4 6 8]

b = np.linspace(0, 10, 6)
print(b)
# [0. 2. 4. 6. 8. 10.]
```

– Le module numpy redéfini un certain nombre de fonctions mathématiques. Ainsi, pour tracer la courbe de la fonction $\sin(x)$ sur l'intervale $[0, 10]$ on peut procéder ainsi :

```
x = np.linspace(0, 10, 1000)
y = np.sin(x)
plt.plot(x, y)
```

– Découvrons les trois fonctions spéciales : `np.ones`, `np.zeros`, `np.eye`, ainsi que la fonction `np.diag`.

```
a = np.ones((3, 5))
# [[1. 1. 1. 1. 1.]
#  [1. 1. 1. 1. 1.]
#  [1. 1. 1. 1. 1.]]

b = np.ones((3, 5), np.int)
# [[1 1 1 1 1]
#  [1 1 1 1 1]
#  [1 1 1 1 1]]

c = np.zeros((3, 5))
# [[0. 0. 0. 0. 0.]
#  [0. 0. 0. 0. 0.]
#  [0. 0. 0. 0. 0.]]

d = np.zeros((3, 5), dtype=np.bool)
# [[False False False False False]
#  [False False False False False]
#  [False False False False False]]

e = np.eye(3)
# [[1. 0. 0.]
#  [0. 1. 0.]
#  [0. 0. 1.]]

f = np.diag([1, 2, 3])
# [[1 0 0]
#  [0 2 0]
#  [0 0 3]]
```

– Tableaux aléatoires :

```
# tirage unifrme d'entiers dans [0, 10[
a = np.random.randint(0, 10, size=(2, 5))
print(a)
# [[8 2 6 2 7]
#  [4 3 5 5 5]]

# (2, 5) échantillons suivant une distribution binomiale (10, .3)
```

```python
b = np.random.binomial(10, .3, (2, 5))
print(b)
# [[1 1 2 3 3]
#  [1 2 4 2 1]]

# 5 échantillons suivant une distribution geometrique (.3)
c = np.random.geometric(.3, 5)
print(c)
# [1 1 1 3 3]

# 5 échantillons suivant une distribution de poisson (4.3)
d = np.random.poisson(4.3, 5)
print(d)
# [2 6 6 3 3]
```

– Un tableau possède des attributs clés : `size` qui donne le nombre d'éléments, `shape` qui renvoie le tuple du format et `ndim` qui indique le nombre d'indice nécessaires au parcours du tableau (i.e. le nombre d'éléments dans le tupple).

```python
a = np.array([[1, 2, 3], [4, 5, 6]])
print(a.size)   # 6
print(a.shape)  # (2, 3)
print(a.ndim)   # 2
```

– L'attribut `shape` est mutable : on peut redimensionner un tableau via la méthode `reshape()`.

```python
a.shape = (3, 2) # ou a = a.reshape((3, 2))
print(a)
# [[1 2]
#  [3 4]
#  [5 6]]

a.shape = (6,) # ou a.shape = (-1,)
print(a)
# [1 2 3 4 5 6]
```

– Le tranchage (slicing) s'étend aux tableaux multidimensionnels en séparant les intervalles des différentes dimensions par des virgules. Il est important de noter que cette opération renvoie une vue et non une copie indépendante.

```python
a = np.array([[10, 11, 12, 13],
              [20, 21, 22, 23],
              [30, 31, 32, 33]])

# sous-tableau : lignes d'indices 0 à 1, colonnes d'indices 1 à 2
print(a[0:2, 1:3])
# [[11 12]
#  [21 22]]

print(a[:, 2])     # toutes les lignes, colonne d'indice 2
# [12 22 32]

print(a[1, :])     # ligne d'indice 1, toutes les colonnes
# [20 21 22 23]
```

– Les masques (indexation booléenne) permettent de sélectionner ou modifier des éléments validant une condition logique. Si `a` est un tableau `numpy`, et `b` un tableau de booléens de même format, alors `a[b]` met en relation un à un les éléments de `a` et ceux de `b`, en ne conservant que les éléments de `a` associés à la valeur `True`.

```python
a = np.arange(10)
# [0 1 2 3 4 5 6 7 8 9]

# extraction des multiples de 3
print(a[a % 3 == 0])
```

```
# [0 3 6 9]

# remplacement des valeurs paires par -1
a[a % 2 == 0] = -1
print(a)
# [-1  1 -1  3 -1  5 -1  7 -1  9]

# mecanisme sous-jacent
mask = a > 5
print(mask)
# [False False False False False False True True True True]
print(a[mask])    # extraction des elements correspondant aux True
# [6 7 8 9]
```

7.3 Opérations sur les tableaux numpy

- Les opérations arithmétiques usuelles (`+`, `*`, `**`...) s'appliquent terme à terme.
- Des méthodes statistiques sont disponibles : `max`, `min`, `sum`, `prod`, `mean` (moyenne arithmétique), `var` (variance), `std` (écart-type).
- Pour l'algèbre linéaire, le produit matriciel se fait avec `np.dot(a, b)` ou `a.dot(b)`. Le produit scalaire canonique utilise `np.vdot` et le produit vectoriel `np.cross`. La transposée s'obtient avec `.T` ou `.transpose()`.
- Le sous-module `np.linalg` fournit des outils avancés : `solve` (résolution de système), `inv` (inverse), `det` (déterminant), `norm` (norme) et `eig` (éléments propres).
- La classe `Polynomial` (du module `numpy.polynomial`) permet de manipuler formellement des polynômes (racines, dérivées, primitives) définis par leurs coefficients.

Chapitre 8

Matplotlib

8.1 Tracé de courbes

- Le sous-module `matplotlib.pyplot` trace des courbes par interpolation linéaire entre des points discrets. Il nécessite en entrée deux séquences à une dimension, de même taille. La qualité du rendu visuel dépend directement du nombre de points calculés.
- Pour générer ces séquences, on distingue l'approche utilisant les listes Python, et celle utilisant les tableaux `numpy`. Cette dernière offre des fonctions dédiées pour les intervalles et permet d'appliquer directement des opérations mathématiques sur l'ensemble des valeurs.

```
import math # souvent necessaire
import matplotlib.pyplot as plt

x = list(range(10)) # uniquement des entiers

y1 = [math.sin(val) for val in x] # par comprehension

y2 = [] # avec une boucle
for val in x:
    y2.append(math.sin(val))
```

```
import numpy as np
import matplotlib.pyplot as plt

# points de 0 inclu a 10 exclu, avec un pas de 0.1
x1 = np.arange(0, 10, 0.1)

# 100 points lineairement espaces entre 0 et 10 inclus
x2 = np.linspace(0, 10, 100)
# remarque : le pas n'est donc pas 0.1

# calcul direct sur l'ensemble du tableau
y = np.sin(x2)
# ou encore y = 2*x etc. fonctionne, contrairement aux listes
```

- La fonction la plus importante est `plot(listeX, listeY, options)`. Les options sont facultatives, mais permettent de personnaliser l'aspect visuel du graphique, notamment la couleur (p. ex. `r`, `g`, `b`), la forme des marqueurs (p. ex. `o`, `s` (carré), `+`) et le style de la ligne (p. ex. `-`, `--`, `:`). Ces options sont souvent regroupées dans une chaîne de caractères courte (p. ex. `'ro--'` i.e. une ligne rouge en pointillés avec des marqueurs circulaires).
- L'exécution de `plot` ne produit aucun affichage immédiat : les commandes graphiques s'accumulent silencieusement dans la figure courante. Il est impératif d'utiliser la fonction `plt.show()` en fin de script pour déclencher le rendu et ouvrir la fenêtre.
- L'habillage et la gestion de la fenêtre s'effectuent via diverses commandes complémentaires :
  - `plt.figure()` initialise une nouvelle fenêtre vierge (utile pour générer plusieurs graphiques indépendants, par exemple `figure(2)`, `figure("Vitesse")`, ou juste `figure()` qui incrémente

automatiquement). D'autres paramètres existent (`figsize`, `dpi`, `facecolor`). Si cette fonction n'est pas appelée explicitement, Matplotlib crée automatiquement une figure par défaut dès la première commande de tracé rencontrée.

- `plt.clf()` efface le contenu de la figure courante sans fermer la fenêtre.
- `plt.xlabel("Texte")` et `plt.title("Texte")` ajoutent les étiquettes d'axes et le titre.
- `plt.grid()` superpose une grille de lecture en arrière-plan.
- `plt.legend()` affiche l'encadré de légende (requiert l'argument `label` dans `plot`).
- `plt.axis([xmin, xmax, ymin, ymax])` : force manuellement les limites d'affichage des axes.
- `plt.axis('equal')` impose un repère orthonormé.

– Bien que `plot` soit d'un usage général, d'autres fonctions répondent à des besoins graphiques spécifiques : `plt.scatter` pour des nuages de points non reliés, `plt.semilogx` ou `plt.loglog` pour les échelles logarithmiques, et `plt.hist` pour les histogrammes.

Chapitre 9

Récursivité

9.1 Principe

– Une fonction est dite *récursive* si son corps contient un ou plusieurs appels à elle-même. C'est une méthode de résolution de problèmes consistant à décomposer un problème complexe en sous-problèmes de même nature mais de taille réduite.

– Pour qu'une fonction récursive soit valide et se termine, deux conditions sont impératives :
  • L'existence d'un ou plusieurs cas de base (ou conditions d'arrêt) qui sont traités sans appel récursif.
  • Une progression stricte des appels récursifs vers ce cas de base (souvent via un paramètre entier décroissant ou la taille d'une liste qui diminue).

– L'exemple canonique est le calcul de la factorielle $n! = n \times (n-1)!$ avec $0! = 1$.

```
1 def factorielle(n):
2     if n == 0:          # cas de base
3         return 1
4     else:               # appel recursif
5         return n * factorielle(n - 1)
```

9.2 Analyse des fonctions récursives

– Lorsqu'une fonction s'appelle elle-même, l'interpréteur suspend l'exécution courante et empile le contexte (variables locales, paramètres, adresse de retour) dans une structure de données appelée *pile d'exécution* (call stack). Lorsque le cas de base est atteint, les résultats sont renvoyés successivement (« dépilés ») lors de la remontée. Si la profondeur de récursion est trop importante, on risque une erreur de type `RecursionError` (débordement de pile ou *stack overflow*).

– Une fonction est dite *récursive terminale* si l'appel récursif est la toute dernière instruction exécutée. C'est qu'il n'y a pas de « remontée ».

– Tout programme récursif peut être transformé en version itérative (avec des boucles).
  • Avantages du récursif : Code souvent plus élégant, lisible et proche de la définition mathématique (ex : suites, structures arborescentes).
  • Inconvénients : Coût mémoire (pile) et surcoût temporel lié aux appels de fonctions. L'itératif est généralement plus efficace en Python.

– Exemple de la suite de Fibonacci : la définition mathématique est $F_0 = 0$, $F_1 = 1$ et $F_n = F_{n-1} + F_{n-2}$. L'implémentation récursive « naïve » est extrêmement inefficace car elle recalcule de nombreuses fois les mêmes termes.

```
1         def fibo_rec(n):
2             if n < 2: return n
3             return fibo_rec(n-1) + fibo_rec(n-2)
```

Une version itérative est indispensable pour calculer des termes de rang élevé.

Chapitre 10

Matrices de pixels et images

10.1 Formats de représentation d'image

- Il existe deux modes de codage numérique. Le mode *vectoriel* décrit les formes par des propriétés mathématiques (zoom infini sans perte, ex : PDF, SVG). Le mode *matriciel* (ou *bitmap*) repose sur une grille de pixels (ex : PNG, JPEG).
- Une image matricielle se caractérise par sa définition (nombre de pixels), sa résolution (nombre de pixels par unité de longueur, en dpi ou ppp) et aussi par sa quantification des couleurs exprimée en bits par pixel (p. ex. noir et blanc equivaut à 1bpp ; 256 nuances de gris equivaut à 8bpp ; 256 nuances dans les trois composantes equivaut à 24bpp...).
- Généralement, la colorimétrie utilise le modèle RVB (Rouge, Vert, Bleu). Chaque couleur est codée sur un octet (de 0 à 255). Un pixel est ainsi un triplet (R, V, B). Le noir correspond à $(0, 0, 0)$ et le blanc à $(255, 255, 255)$.

10.2 Manipulation des matrices de pixels

- L'étude des images matricielles s'effectue ici avec le module `matplotlib.image`, bien qu'il existe en Python d'autres bibliothèques plus ou moins similaires.
- On utilise `pyplot` pour l'affichage.
- Dans notre cas, une image est traitée comme un tableau Numpy de dimension 3 (hauteur, largeur, composantes).

```
import numpy as np
import matplotlib.image as img
import matplotlib.pyplot as plt

im = img.imread("image.png")  # chargement et stockage dans la variable
plt.imshow(im)                # préparation de l'affichage
plt.show()                    # affichage

print(im.shape)               # dimensions de l'image
print(im)                     # contenu du tableau

# création d'une image vide ; attention double parenthèse et h * l
image = np.zeros((1080, 1920, 3))
image[342, 135] = [100, 50, 12]  # modification d'un pixel
```

10.3 Transformation d'images

- Symétrie axiale (ici sur l'axe vertical)

```
H, L, C = im.shape
im_sym = np.zeros((H, L, C))
```

```
3
4  # méthode iterative
5  for i in range(H):
6      for j in range(L):
7          im_sym[i, j] = im[i, L - 1 - j]
8
9  # méthode par slicing
10 im_sym = im[:, ::-1]
```

– Rotation (90°)

```
1  im_rot = np.zeros((L, H, C)) # dimensions inversees
2
3  # méthode iterative
4  for i in range(H):
5      for j in range(L):
6          im_rot[L - 1 - j, i] = im[i, j]
```

– Passage en niveau de gris (par moyenne pondérée)

```
1  H, L, C = im.shape
2  im_gris = np.zeros((H, L)) # tableau 2d (pas de 3e dimension)
3
4  # méthode iterative
5  for i in range(H):
6      for j in range(L):
7          r, v, b = im[i, j] # si C = 3
8          im_gris[i, j] = 0.2125*r + 0.7154*v + 0.0721*b # norme 709
9          # sinon, (r+v+b)/3
10
11 # affichage d'une image en niveau de gris
12 plt.imshow(im_gris, cmap='gray')
13 plt.show()
```

10.4 Modification par convolution : filtrage

– Le filtrage consiste à modifier la valeur d'un pixel en fonction de ses voisins grâce à une petite matrice.

– Le nouveau pixel est obtenu par un filtre (ou produit de convolution) permettant pour chaque pixel de modifier sa valeur en fonction des valeurs des pixels avoisinants, affectées de coefficients. Pour conserver la luminosité, on divise souvent le résultat par la somme des coefficients du filtre.

– Pour un pixel en bordure, certains voisins manquent. Une solution consiste à ignorer les bords de l'image (balayage de la ligne 1 à $h-1$).

– On peut appliquer un filtre sur une zone spécifique en utilisant une matrice masque (image binaire). Le résultat final est une combinaison : $I_{finale} = M \cdot I_{floue} + (1-M) \cdot I_{initiale}$.

– Flou (moyenneur)

```
1  n = 3
2  H, L, C = im.shape
3  b = n//2 # bordure
4
5  im_flou = np.zeros((H, L, C))
6
7  for i in range(b, H - b):
8      for j in range(b, L - b):
9          for c in range(C):
10             s=0
11             for k in range(n):
12                 for l in range(n):
13                     s = s + im[i+k-n//2, j+l-n//2, c]
14             im_flou[i, j, c]=s/(n*n)
```

10.5 Détection de contours

- La détection de contour repose sur l'identification des changements brutaux de couleur ou de contraste entre pixels voisins.
- On calcule pour chaque pixel une « distance » euclidienne par rapport à ses voisins directs de valeurs p_1, p_2, p_3 et p_4 (image en niveau de gris pour avoir une seule valeur). Une formule courante est la distance euclidienne des différences : $d = \sqrt{(p_1 - p_3)^2 + (p_2 - p_4)^2}$.
- On compare cette distance à une valeur seuil : si $d > seuil$, le pixel appartient à un contour et est tracé en blanc ; sinon, il est laissé en noir.

Chapitre 11

Algorithmes gloutons et dichotomiques

11.1 Algorithmes gloutons

- Les *algorithmes gloutons* s'inscrivent dans le cadre de la résolution de problèmes d'optimisation. Un problème d'optimisation consiste à choisir, parmi un ensemble de solutions possibles, celle qui maximise un gain : la *fonction-objectif* ; ou minimise un coût : la *fonction-coût*.
- L'ensemble des solutions qui respectent les contraintes imposées par le problème est appelé l'*ensemble admissible*. Une solution qui répond au critère d'optimisation (le meilleur score possible) est une *solution globale*.
- Quelques exemples classiques de problèmes d'optimisation :
  - Le problème du sac à dos (maximiser la valeur des objets emportés sous contrainte de poids).
  - Le problème du rendu de monnaie (minimiser le nombre de pièces pour atteindre une somme).
  - Le problème du voyageur de commerce (minimiser la distance pour visiter un ensemble de villes).
- Le principe d'un algorithme glouton est de faire, à chaque étape, le choix qui semble le meilleur localement (le choix optimal à l'instant t), sans jamais revenir sur une décision prise, dans l'espoir que cette suite de choix locaux mène à l'optimum global. Notons bien sûr que cela ne garantit pas toujours de trouver la solution optimale absolue, mais fournit souvent une solution approchée acceptable rapidement.

11.2 Algorithmes dichotomiques

- La méthode *dichotomique* (du grec « couper en deux ») est une stratégie de recherche efficace qui consiste à réduire de moitié l'espace de recherche à chaque étape.
- Voyons deux applications : la recherche dans une liste triée et la recherche de racine d'une fonction monotone.
- Recherche dans une liste triée : On cherche un élément x_0 dans une liste L triée. On compare x_0 avec l'élément central de la liste. Si x_0 est plus petit, on ne cherche que dans la moitié gauche ; s'il est plus grand, dans la moitié droite. On répète le processus jusqu'à trouver l'élément ou épuiser la liste.

```
1  def recherche_dichotomie(L, x0):
2      """ Recherche x0 dans une liste L triee. Renvoie un booleen. """
3      g = 0                # indice gauche
4      d = len(L) - 1       # indice droit
5      found = False
6
7      while g <= d and not found:
8          m = (g + d) // 2  # indice milieu
9          if x0 == L[m]:
10             found = True
11         elif x0 < L[m]:
12             d = m - 1
```

```
        else:
            g = m + 1

    return found
```

- Recherche de racine : Il s'agit de trouver une solution approchée de l'équation $f(x) = 0$ sur un intervalle $[a, b]$. Le principe repose sur le Théorème des Valeurs Intermédiaires (TVI). Si f est continue et que $f(a)$ et $f(b)$ sont de signes opposés ($f(a) \cdot f(b) < 0$), alors il existe au moins une racine dans l'intervalle.
- L'algorithme calcule le milieu $m = \frac{a+b}{2}$. Si $f(a)$ et $f(m)$ sont de signes contraires, la racine est dans $[a, m]$, sinon elle est dans $[m, b]$. On réduit ainsi la taille de l'intervalle par 2 à chaque itération jusqu'à ce que sa largeur soit inférieure à une précision ϵ donnée.

```
def zero_dichotomie(f, a, b, epsilon):
    """ Trouve une racine de f dans [a, b] avec precision epsilon """
    val_g = a
    val_d = b
    while (val_d - val_g) > epsilon:
        m = (val_g + val_d) / 2
        if f(val_g) * f(m) <= 0:  # changement de signe a gauche
            val_d = m
        else:                     # changement de signe a droite
            val_g = m
    return (val_g + val_d) / 2
```

Chapitre 12

Algorithmes de tri

12.1 Introduction

- Un algorithme de tri a pour vocation d'ordonner les éléments d'une liste selon une relation d'ordre préétablie.
- Un algorithme est dit *en place* (in-place) s'il opère directement sur la structure séquentielle en mémoire, ne requérant qu'un espace supplémentaire constant, indépendamment de la taille des données traitées. Pour ces fonctions de tri, il n'est donc pas nécessaire de renvoyer la liste d'entrée, et il faut en prévoir en amont une copie si besoin.
- Un tri *comparatif* fonde sa logique exclusivement sur la comparaison des éléments deux à deux.
- La *stabilité* désigne la propriété de préserver l'ordre relatif initial des éléments considérés comme égaux.
- Le langage Python propose deux méthodes natives de tri :

```
L = [3, 1, 2]
L.sort()  # L est modifiée en place et devient [1, 2, 3]
L = [3, 1, 2]
L_triee = sorted(L)  # L reste [3, 1, 2], L_triee est [1, 2, 3]
```

  `L.sort()` est une méthode réservée aux listes, alors que `sorted(L)` est une fonction universelle qui accepte n'importe quel itérable (liste, tuple, chaîne de caractères, dictionnaire).
- L'efficacité d'un algorithme s'évalue par sa *complexité temporelle*, notée $O(f(n))$. Cela représente l'ordre de grandeur du nombre d'opérations nécessaires pour trier une liste de taille n. Concrètement, un tri en $O(n^2)$ sera très lent pour de grandes listes (si n double, le temps est multiplié par 4), tandis qu'un tri en $O(n \log n)$ restera performant.
- Remarques :
  - Dans la suite, on se focalisera sur le tri dans l'ordre croissant d'entiers.
  - Il existe à chaque fois plusieurs variantes pour coder un même algorithme.

12.2 Tri par sélection

- Le principe consiste, à chaque étape i, à parcourir la partie non triée de la liste (à droite) pour identifier le minimum, puis à l'échanger avec l'élément situé à la position i. La sous-liste triée croît ainsi progressivement de la gauche vers la droite.
- Ce tri est en place mais généralement instable, car un échange peut modifier l'ordre relatif d'éléments égaux.

```
def tri_selection(L):
    for i in range(len(L) - 1):
        i_min = i

        # recherche d'un meilleur candidat
```

```
        for k in range(i + 1, len(L)):
            if L[k] < L[i_min]:
                i_min = k

        # échange
        if i_min != i: # facultatif
            L[i], L[i_min] = L[i_min], L[i]
```

– La complexité temporelle est quadratique, soit $O(n^2)$, quel que soit l'arrangement initial des données (pire, meilleur et moyen cas), car le parcours intégral de la sous-liste restante est systématique.

12.3 Tri par insertion

– Analogue au tri manuel d'un jeu de cartes, cet algorithme parcourt la liste de gauche à droite. L'élément courant (la clé) est inséré à sa position légitime dans la sous-liste gauche déjà triée, après décalage des éléments supérieurs.

– Ce tri est en place et stable.

```
def tri_insertion(L):
    for i in range(1, len(L)):
        cle = L[i]
        k = i - 1

        # décalage des elements plus grands que la cle
        while k >= 0 and L[k] > cle:
            L[k + 1] = L[k]
            k -= 1

        # insertion
        L[k + 1] = cle
```

– La complexité dans le pire cas (liste triée à l'envers) est quadratique en $O(n^2)$. Dans le meilleur cas (liste déjà triée), elle devient linéaire en $O(n)$, car la boucle de décalage n'est jamais exécutée.

12.4 Tri par fusion

– Cet algorithme applique le paradigme « diviser pour régner ». La liste est scindée en deux moitiés égales, triées récursivement, puis fusionnées pour constituer la liste finale ordonnée.

– Ce tri est stable, mais ne s'effectue généralement pas en place (nécessité de mémoire auxiliaire pour la fusion).

```
def fusion(L1, L2):
    """ Fusionne deux listes triees en une seule """
    res = []
    i = j = 0
    while i < len(L1) and j < len(L2):
        if L1[i] <= L2[j]:
            res.append(L1[i])
            i += 1
        else:
            res.append(L2[j])
            j += 1
    return res + L1[i:] + L2[j:] # ajout des restes

def tri_fusion(L):
    if len(L) <= 1:
        return L
    else:
        m = len(L) // 2
        g = tri_fusion(L[:m])
        d = tri_fusion(L[m:])
```

```
21          return fusion(g, d)
```

– La complexité temporelle est toujours en $O(n \log n)$, la profondeur de récursion étant logarithmique et le coût de la fusion linéaire.

12.5 Tri par comptage

– Ce tri se distingue car il ne compare pas les éléments entre eux mais compte leurs occurrences. Il nécessite de connaître au préalable le maximum (ou un majorant) (et éventuellement le minimum) des valeurs de la liste, sinon de le calculer en début de fonction.

– Le tri par comptage ne s'effectue pas en place.

– Dans le cas général, il est efficace uniquement si les valeurs sont des entiers positifs compris dans un intervalle raisonnable.

```
1  def tri_com(L):
2
3      maxi = max(L)
4
5      F = [0] * (maxi + 1) # liste des fréquences
6
7      for k in L: # comptage
8          F[k] += 1
9
10     i = 0
11     for k in range(maxi + 1):
12         for _ in range(F[k]):
13             L[i] = k
14             i += 1
```

– La complexité est linéaire en $O(n + m)$, où n est la taille de la liste et m la valeur maximale. C'est extrêmement rapide si m est proche de n, mais catastrophique si m est très grand.

– Voici une version capable de traiter des entiers négatifs :

```
1  def tri_comptage(L):
2      mini = min(L)
3      maxi = max(L)
4
5      F = [0] * (maxi - mini + 1)
6
7      for k in L:
8          F[k - mini] += 1
9
10     i = 0
11     for k in range(len(F)):
12         for _ in range(F[k]):
13             L[i] = k + mini
14             i += 1
```

– On peut également adapter ce tri aux nombres décimaux (à condition que la précision soit bornée) en les multipliant par une puissance de 10 suffisante pour les transformer en entiers, appliquer l'algorithme précédent, puis rétablir les valeurs initiales.

12.6 Tri rapide

– Également basé sur la récursivité, cet algorithme choisit un élément nommé *pivot*. Une étape de partitionnement réorganise la liste (fonction annexe ou non) : les éléments inférieurs au pivot passent à gauche, les supérieurs à droite. Le pivot est alors correctement placé dans la liste actuelle.

– Voici une première proposition qui n'est pas en place mais stable :

```python
def tri_rapide(L):

    if len(L) <= 1:
        return L

    piv = L[0]
    inf = []
    sup = []

    for e in L[1:]:
        if e < piv:
            inf.append(e)
        else:
            sup.append(e)

    return tri_rapide(inf) + [piv] + tri_rapide(sup)
```

De manière plus élégante, et pour éviter le pire cas systématique :

```python
import random

def tri_rap(L):
    if len(L) <= 1:
        return L

    piv = random.choice(L)
    inf = [e for e in L if e < piv]
    egal = [e for e in L if e == piv]
    sup = [e for e in L if e > piv]

    return tri_rap(inf) + egal + tri_rap(sup)
```

– Voici une seconde version, en place mais instable.

```python
def partition(L, g, d):
    """ Range les elements par rapport au pivot """
    pivot = L[g] # première valeur
    front = g # frontière entre les inf. et les sup. au pivot

    # parcours de la zone de g+1 jusqu'a d-1
    for i in range(g + 1, d):
        if L[i] < pivot: # si plus petit
            front += 1 # décalage du rang de la frontière
            # placement à gauche de la frontiere
            L[i], L[front] = L[front], L[i]

    # positionnement du pivot à sa place définitive : front
    L[g], L[front] = L[front], L[g]
    return front # renvoie la position du pivot

def tri_rapide(L, g=0, d=None): # g et d ne sont pas requis initialement
    """ Fonction principale recursive """
    # au premier appel, on travaille sur toute la liste
    if d is None:
        d = len(L)

    # s'il reste au moins 2 elements à trier dans la zone
    if g < d - 1:
        piv = partition(L, g, d)  # placement du pivot
        tri_rapide(L, g, piv)     # trie de la partie gauche
        tri_rapide(L, piv + 1, d) # trie de la partie droite
```

– La complexité moyenne est excellente en $O(n \log n)$. Cependant, dans le pire cas (si le pivot est mal choisi, par exemple le minimum sur une liste déjà triée), elle dégrade en $O(n^2)$.

12.7 Tri à bulle

- Le tri à bulle consiste à parcourir plusieurs fois la liste en comparant les éléments adjacents. Lorsque deux éléments consécutifs sont dans le mauvais ordre, ils sont échangés.
- À chaque passage, les plus grandes valeurs « remontent » progressivement vers la fin de la liste, comme des bulles qui montent à la surface.
- Ce tri est en place et stable.

```python
def tri_bulle(L):
    n = len(L)

    # nombre de passages
    for i in range(n - 1):

        # parcours de la zone non triee
        for k in range(n - 1 - i):

            # echange si les elements sont mal ordonnes
            if L[k] > L[k + 1]:
                L[k], L[k + 1] = L[k + 1], L[k]
```

- Exemple sur la liste `[5, 2, 4, 1]` :
  - Premier passage :
  $$[5, 2, 4, 1] \to [2, 5, 4, 1] \to [2, 4, 5, 1] \to [2, 4, 1, 5]$$
  Le plus grand élément est désormais bien placé.
  - Deuxième passage :
  $$[2, 4, 1, 5] \to [2, 1, 4, 5]$$
  - Troisième passage :
  $$[2, 1, 4, 5] \to [1, 2, 4, 5]$$

- Une amélioration classique consiste à arrêter l'algorithme dès qu'aucun échange n'est effectué pendant un passage.
- La complexité temporelle dans le pire et le cas moyen est quadratique en $O(n^2)$.
- Bien que simple à comprendre et à programmer, ce tri est peu efficace sur de grandes listes et reste principalement utilisé à des fins pédagogiques.

Chapitre 13

Complexité algorithmique

13.1 Performances et vocabulaire

– L'évaluation des performances d'un programme dépend de facteurs matériels (architecture du processeur, mémoire) et logiciels (langage compilé ou interprété). Cependant, lorsque le volume de données devient très important, l'efficacité intrinsèque de l'algorithme choisi devient le facteur dominant.

– L'étude de cette efficacité s'articule autour de deux mesures fondamentales, exprimées en fonction de la taille n des données en entrée :
  • La *complexité temporelle* évalue le nombre d'opérations élémentaires (affectations, comparaisons, calculs arithmétiques) exécutées par l'algorithme.
  • La *complexité spatiale* mesure l'empreinte mémoire, c'est-à-dire le nombre d'emplacements mémoire supplémentaires nécessités par l'exécution de l'algorithme.

– Le temps d'exécution pouvant varier pour des données de même taille selon leur disposition (par exemple, une liste déjà triée ou non), l'analyse se concentre généralement sur la complexité dans le pire des cas, qui garantit un temps d'exécution maximal.

– Pour comparer les algorithmes, la notation de Landau, notée $O(f(n))$, est employée. Elle caractérise l'ordre de grandeur asymptotique. Les principales classes de complexité, de la plus performante à la moins efficace, sont :
  • $O(1)$: complexité *constante*. Le temps d'exécution est indépendant de la taille des données.
  • $O(\log n)$: complexité *logarithmique*. Typique des algorithmes divisant le problème par deux à chaque étape (dichotomie).
  • $O(n)$: complexité *linéaire*. Typique d'un parcours simple d'une séquence.
  • $O(n \log n)$: complexité *quasi-linéaire*. Caractéristique des algorithmes de tri optimisés (tri fusion, tri rapide).
  • $O(n^2)$: complexité *quadratique*. Souvent issue de deux boucles imbriquées parcourant les données.
  • $O(2^n)$ et $O(n!)$: complexités *exponentielle* et *factorielle*. Ces algorithmes deviennent impraticables même pour des valeurs modestes de n.

13.2 Évaluation avec des boucles itératives

– L'évaluation de la complexité temporelle nécessite de dénombrer les passages dans les boucles. Pour des boucles `for` simples et successives, les complexités s'additionnent. Le terme prépondérant dicte alors la complexité globale.

– Dans le cas de boucles imbriquées, le nombre de répétitions se multiplie. L'exemple suivant illustre un coût quadratique, où l'instruction d'incrémentation est exécutée $3n^2$ fois. La constante multiplicative est ignorée dans la notation asymptotique, donnant une complexité en $O(n^2)$.

```
1 def mystere1(n):
2     s = 0
3     for i in range(n):
4         for j in range(n):
```

```
            for k in range(3):
                s += 1
    return s
```

– La taille de la boucle intérieure peut dépendre de l'indice de la boucle extérieure. Il est alors nécessaire de recourir aux sommes mathématiques usuelles pour déterminer le nombre exact d'opérations.

```
def mystere2(n):
    s = 0
    for i in range(1, n + 1):
        for j in range(i):
            s += 1
    return s
```

Dans ce second exemple, le nombre total d'itérations correspond à la somme des premiers entiers, ce qui aboutit également à une complexité quadratique en $O(n^2)$:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}.$$

13.3 Évaluation avec `while` et récursivité

– L'analyse de la complexité d'une boucle non bornée (`while`) ou d'une fonction récursive requiert l'établissement d'une relation de récurrence. L'objectif est d'exprimer le coût $C(n)$ pour une entrée de taille n en fonction du coût des étapes précédentes.

– Dans le cas d'une recherche dichotomique, la taille de l'espace de recherche est divisée par deux à chaque itération. L'équation de récurrence s'écrit alors $C(n) = C(n/2) + a$, où a représente le coût constant des opérations élémentaires de l'étape courante (comparaisons, calcul du milieu et affectations).

– Pour résoudre explicitement cette équation, on effectue le changement de variable $n = 2^i$, ce qui équivaut à poser $i = \log_2(n)$. La relation devient $C(2^i) = C(2^{i-1}) + a$. On reconnaît ici la définition d'une suite arithmétique de raison a. Le terme général est donc proportionnel à i. En remplaçant i, on démontre rigoureusement une complexité temporelle logarithmique en $O(\log_2 n)$.

– Une vigilance stricte doit être accordée aux complexités spatiale et temporelle lors de l'implémentation d'algorithmes récursifs manipulant des séquences. L'utilisation du mécanisme de tranchage (*slicing*), tel que l'instruction `L[:m]`, génère une copie physique de la sous-liste en mémoire. Le coût en temps et en espace de cette instruction est directement proportionnel à la taille de la portion copiée.

– En isolant la composante spatiale, l'équation de récurrence s'écrit $C_e(n) = C_e(n/2) + \frac{n}{2}$. Avec le même changement de variable $n = 2^i$, on obtient $C_e(2^i) = C_e(2^{i-1}) + \frac{1}{2}2^i$. L'évaluation de cette expression fait apparaître la somme des termes d'une suite géométrique de raison 2, dont le résultat final est de l'ordre de 2^i, soit n.

– La complexité spatiale, tout comme la complexité temporelle, se dégrade par conséquent en $O(n)$, annulant l'avantage de la dichotomie. Il est donc impératif de travailler directement sur la liste d'origine en transmettant des indices de position (`g` et `d`) en tant que paramètres de la fonction.

13.4 Complexité des opérations natives Python

– L'écriture de programmes performants exige la connaissance préalable du coût asymptotique des opérations masquées par les méthodes natives du langage. Le tableau suivant récapitule les complexités temporelles dans le pire des cas, ou en moyenne amortie, pour les listes (`list`) de taille n et m :

Opération sur une liste	**Complexité temporelle**
Longueur `len(L)`	$O(1)$
Accès en lecture ou écriture `L[i]`	$O(1)$
Ajout en fin `L.append(elt)`	$O(1)$
Retrait du dernier élément `L.pop()`	$O(1)$
Retrait d'un élément d'indice négatif `L.pop(-k)`	$O(k)$
Test de présence `elt in L`	$O(n)$
Copie `L.copy()` ou tranchage `L[:]`	$O(n)$
Extension `L1.extend(L2)`	$O(m)$
Concaténation `L1 + L2`	$O(n+m)$
Recherche du `max(L)` ou `min(L)`	$O(n)$
Tri `L.sort()` ou `sorted(L)`	$O(n \log n)$

– Pour le type `dict`, qui est implémenté sous la forme optimisée d'une table de hachage, l'efficacité est maximale. Les opérations usuelles s'effectuent en temps constant en moyenne, indépendamment de la taille globale du dictionnaire :

Opération sur un dictionnaire	**Complexité temporelle**
Longueur `len(d)`	$O(1)$
Accès en lecture ou écriture `d[k]`	$O(1)$
Ajout d'une nouvelle clé `d[k] = v`	$O(1)$
Test de présence d'une clé `k in d`	$O(1)$
Retrait d'un élément `d.pop(k)` ou `del d[k]`	$O(1)$
Copie intégrale `d.copy()`	$O(n)$

Chapitre 14

Piles et files

14.1 Piles (LIFO)

- Une *pile* est une structure de données linéaire fonctionnant selon le principe LIFO (*Last In, First Out*). On peut l'assimiler à une pile d'assiettes.
- Les opérations élémentaires consistent à *empiler* un élément au sommet, *dépiler* (retirer et renvoyer le sommet), et vérifier si la structure est vide.
- En Python, bien qu'une liste classique puisse suffire, il est rigoureux et optimisé d'utiliser l'objet `deque` du module natif `collections`. Les opérations d'ajout et de retrait s'effectuent alors à temps constant, soit $O(1)$.
- Voici une fonction chargée de copier une pile en $O(n)$:

```python
from collections import deque

def copy_pile(p):
    tmp = deque()
    copy = deque()

    # renversement temporaire
    while p: # equivaut a while len(p) > 0
        tmp.append(p.pop())

    # restauration de la pile originale et construction de la copie
    while tmp:
        e = tmp.pop()
        p.append(e)
        copy.append(e)

    return copy
```

- Voyons une fonction qui compte les éléments d'une pile ($O(n)$). Une implémentation similaire à la fonction précédente est possible. Étudions ici une approche récursive :

```python
def hauteur_pile(p):
    if not p: # pile vide
        return 0
    else:
        v = p.pop() # depile le sommet
        h = 1 + hauteur_pile(p) # hauteur de la pile restante
        p.append(v) # remet le sommet en place
        return h
```

- Une autre fonction pourrait être chargée de renverser l'ordre d'une pile : pour ce faire il faudrait la copier et la renverser, ce qui se fait en trois boucles si l'on souhaite conserver la variable originale.
- Un exemple d'application est la vérification du bon parenthésage d'un texte. Le principe consiste à empiler toute ponctuation ouvrante, et à dépiler lors de la rencontre d'une fermeture pour vérifier

la correspondance.

```
def verif(texte):
    p = deque()

    for c in texte:
        if c in "([{":
            p.append(c)

        elif c in ")]}":
            if not p:
                return "fermeture orpheline"

            ouv = p.pop()

            if ((c == ')' and ouv != '(')
                or (c == ']' and ouv != '[')
                or (c == '}' and ouv != '{')):
                return "discordance"

    if p:
        return "ouverture orpheline"

    return "ok"
```

14.2 Files (FIFO)

- Une *file* fonctionne selon le principe FIFO (*First In, First Out*), à l'image d'une file d'attente.
- Les opérations consistent à *enfiler* un élément en queue et à *défiler* la tête.
- Afin de conserver une complexité en temps constant, l'extraction s'effectue cette fois avec la méthode `popleft()`.
- La fonction de copie d'une file, de complexité $O(n)$, s'écrit de la manière suivante :

```
def copy_file(f):
    tmp = deque()
    copy = deque()

    while f:
        e = f.popleft()
        tmp.append(e)
        copy.append(e)

    while tmp:
        f.append(tmp.popleft())

    return copy
```

- La détermination de la longueur peut s'effectuer par un vidage suivi d'une restauration :

```
def longueur_file(f):
    tmp = deque()
    cpt = 0

    while f:
        tmp.append(f.popleft())
        cpt += 1

    while tmp:
        f.append(tmp.popleft())

    return cpt
```

– Enfin, une procédure permettant de renverser l'ordre des éléments d'une file en n'utilisant que les opérations de base nécessite l'emploi de la récursivité :

```
def renverse_file(f):
    if f:
        e = f.popleft()
        renverse_file(f)
        f.append(e)
```

Sinon, on utiliserait une pile, ce qui correspond en réalité au mécanisme sous-jacent de la récursivité.

Chapitre 15

Compléments

15.1 Compléments en vrac :)

– Pour insérer des valeurs de variables au sein d'une chaîne de caractères, il suffit de placer la lettre `f` juste avant les guillemets et d'écrire les noms des variables (ou même des expressions) entre accolades `{}` à l'intérieur de la chaîne.

```
value = 10
print(f"Valeur mesurée : {value}") # Insertion simple
print(f"Résultat : {value + value/3}") # Operations dans les accolades
```

– La fonction `input(message)` permet de mettre le programme en pause et de demander à l'utilisateur de saisir du texte au clavier. Le `message` est affiché pour guider l'utilisateur. La fonction `input` renvoie toujours une chaîne de caractères (`str`), même si l'utilisateur tape des chiffres. Il est donc impératif de convertir (caster) le résultat si l'on attend un nombre.

```
nom = input("Quel est votre nom ? ") # Renvoie un str
age_str = int(input("Quel est votre age ? ")) # Renvoie un int
```

– Le module `random` de la bibliothèque standard est utilisé pour générer des nombres aléatoires scalaires (uniques). Il s'importe via `import random`. Voici quelques fonctions notables :

- `random.random()` : Renvoie un flottant x tel que $0.0 \leq x < 1.0$ (avec beaucoup de digits).
- `random.randint(a, b)` : Renvoie un entier n tel que $a \leq n \leq b$ (bornes incluses !).
- `random.choice(seq)` : Renvoie un élément choisi au hasard dans une séquence non vide (liste, chaîne...).
- `random.shuffle(liste)` : Mélange les éléments d'une liste sur place (ne renvoie rien).

```
import random
de = random.randint(1, 6) # Simule un de à 6 faces
piece = random.choice(["Pile", "Face"])
```

15.2 Fonctions et méthodes (HP)

– Une fonction autonome s'applique à des arguments placés entre parenthèses selon la syntaxe `fonction(arg1, arg2)`. L'outil existe indépendamment des variables qu'il reçoit. Le nombre d'arguments requis dépend uniquement de la définition de la fonction.

```
print()          # génère un saut de ligne

L = [10, 20, 30]
n = len(L)       # mesure la taille de l'objet passé en paramètre

m = max(5, 12, 3) # renvoie l'élément maximal parmi les arguments
```

– Une *méthode* est une fonction intégrée à un type d'objet spécifique. Son appel impose la notation pointée selon la syntaxe `objet.methode(arg1, arg2)`. L'objet placé avant le point est l'acteur

principal qui subit l'action, tandis que les éléments entre parenthèses sont des arguments secondaires. Le nombre d'arguments varie également selon la méthode appelée.

```
L = [1, 2, 3]
x = L.pop()       # retire et renvoie le dernier élément de la liste
L.clear()         # vide intégralement la liste de ses éléments

L.append(4)       # insère la valeur spécifiée à la fin de la liste

c = "alea"
s = c.replace("a", "o", 1) # remplace le premier a par un o
```

– Les deux syntaxes ne sont pas interchangeables. Tenter d'utiliser une méthode sous la forme d'une fonction autonome, ou appliquer une méthode non définie pour un type d'objet donné, provoque une erreur qui interrompt l'exécution du programme.

```
append(L, 5)      # erreur
L.len()           # erreur
```

Deuxième partie

Représentation des nombres

Chapitre 16

Représentation des nombres en mémoire

16.1 Bases de numération

- Le système décimal (base 10) est la norme de référence courante, mais l'architecture matérielle des ordinateurs repose sur le système binaire (base 2), dont les uniques symboles sont 0 et 1.
- L'écriture d'un nombre entier naturel dans une base B se décompose polynomialement. Un nombre représenté par la séquence $a_n a_{n-1} \dots a_1 a_0$ en base B possède une valeur décimale équivalente définie par

$$\sum_{k=0}^{n} a_k B^k,$$

  où chaque chiffre a_k appartient à l'intervalle d'entiers $[0, B-1]$.
- Le système hexadécimal (base 16) est privilégié en électronique numérique car il permet une représentation très compacte des séquences binaires. Il utilise les chiffres de 0 à 9, complétés par les lettres de A à F (de 10 à 15).
- Le passage d'une base à une autre repose sur des méthodes algorithmiques :
  - D'une base B vers la base 10 : Évaluation du polynôme des puissances successives de la base. Par exemple, $BC5_{16} = 11 \times 16^2 + 12 \times 16^1 + 5 \times 16^0 = 3013_{10}$.
  - De la base 10 vers une base B : Divisions euclidiennes successives du quotient par la base B jusqu'à obtenir un quotient nul. La lecture des restes de bas en haut (du dernier au premier) donne l'écriture dans la nouvelle base.
  - Entre les bases 2 et 16 : Une conversion directe est possible en regroupant les bits par paquets de quatre (un quartet ou *nibble*). Par exemple, la lecture du motif $1100\,0000_2$ s'associe directement à $C0_{16}$.
- Le langage Python gère nativement ces différentes bases au moyen de préfixes et de fonctions de conversion spécifiques.

```
# saisie directe via prefixe
valeur_bin = 0b1101  # base 2 (vaut 13)
valeur_hex = 0xA3    # base 16 (vaut 163)

# conversion d'un entier vers une chaine
chaine_bin = bin(13)  # renvoie '0b1101'
chaine_hex = hex(163) # renvoie '0xa3'
```

16.2 Représentation des entiers

- Les opérations au sein du microprocesseur s'effectuent sur des blocs d'information appelés *mots-machine*, dont la taille n s'exprime en bits (généralement 32 ou 64 bits). L'*octet* (8 bits) constitue l'unité usuelle de stockage en mémoire.

– Les entiers naturels sont encodés en binaire direct. Un mot de n bits permet de couvrir l'intervalle d'entiers allant de 0 à 2^n-1. Le dépassement de cette borne supérieure engendre une erreur matérielle par dépassement de capacité (*overflow*). Python, en revanche, gère dynamiquement la taille des entiers en s'affranchissant de cette limite matérielle.

– Le codage des entiers relatifs (signés) s'effectue par la méthode du *complément à 2*. L'intervalle de représentation est translaté pour s'étendre de -2^{n-1} à $2^{n-1}-1$.

– Dans cette représentation, le bit de poids fort indique le signe de la valeur : 0 désigne un entier positif, et 1 un entier négatif.

– L'obtention du code d'un entier négatif a (où $a<0$) sur n bits peut se faire par deux approches :
  • Approche mathématique : Coder en binaire direct la valeur positive résultant de l'opération $a+2^n$.
  • Approche algorithmique : Écrire la valeur absolue de a en binaire, inverser tous les bits (les 0 deviennent 1, et inversement), puis ajouter la valeur 1 au résultat.

16.3 Représentation des nombres réels

– La conversion d'un nombre à virgule en binaire requiert un traitement séparé de sa partie entière et de sa partie fractionnaire.

– La méthode de conversion de la partie fractionnaire consiste à la multiplier successivement par 2. À chaque itération, la partie entière du résultat (nécessairement 0 ou 1) constitue le chiffre binaire suivant de la partie fractionnaire, de gauche à droite. Le processus se répète avec la nouvelle partie fractionnaire jusqu'à ce que celle-ci devienne nulle ou que la limite de la mémoire soit atteinte.

– Les ordinateurs utilisent la représentation en *virgule flottante* normalisée par l'IEEE 754. Cette norme attribue trois rôles distincts aux bits du mot-machine :
  • Le *signe* (s) : 1 bit (0 pour positif, 1 pour négatif).
  • L'*exposant* (e) : Décalé d'une valeur fixe (127 en simple précision sur 8 bits, 1023 en double précision sur 11 bits) pour éviter l'usage d'un exposant signé.
  • La *mantisse* (m) : Représente les chiffres significatifs de la partie fractionnaire.

– La conversion manuelle d'un réel (par exemple $-243,25$) vers la norme IEEE 754 s'opère ainsi :
  • Conversion en binaire complet : $-11110011,01_2$.
  • Décalage de la virgule (normalisation) : $-1,111001101_2 \times 2^7$. L'unique bit 1 avant la virgule est implicite et ne sera pas mémorisé.
  • Calcul de l'exposant décalé : $7+127=134=10000110_2$.
  • La mantisse est complétée par des zéros à droite pour atteindre la taille requise.

– L'évaluation de la valeur x à partir de ses champs (s, e, m) dépend du contenu de l'exposant :
  • *Nombres normalisés* : Si l'exposant n'est ni rempli de zéros ni de uns, la valeur est $x=(-1)^s \times (1,m_1m_2\dots)\times 2^{e-\text{décalage}}$.
  • *Nombres dénormalisés* : Si $e=0$ et $m\neq 0$, le bit implicite devient 0, permettant de coder des nombres extrêmement proches de zéro.
  • *Zéro* : Si $e=0$ et $m=0$. Il existe ainsi un $+0$ et un -0.
  • *Infinis* : Si e est à sa valeur maximale (que des 1) et $m=0$, le nombre représente $\pm\infty$.
  • *NaN (Not a Number)* : Si e est à sa valeur maximale et $m\neq 0$, signale une erreur ou une indétermination mathématique.

```python
# affichage de l'ecriture hexadecimale interne du flottant
valeur = 5.5
print(valeur.hex())
# affiche 0x1.6000000000000p+2
```

– La représentation fractionnaire binaire induit un inconvénient majeur : certains nombres décimaux simples, tels que $0,1$, possèdent un développement binaire infini et périodique. Ils ne peuvent donc pas être stockés de manière exacte et subissent une troncature matérielle.

– Ces approximations s'accumulent lors des opérations arithmétiques, entraînant des erreurs de précision notables. Par exemple, l'évaluation de `0.1 + 0.2` en Python renvoie `0.30000000000000004`.
– Il est par conséquent impératif de ne jamais vérifier l'égalité stricte entre deux nombres flottants avec l'opérateur `==`. La comparaison s'effectue en validant que la valeur absolue de leur différence demeure inférieure à une tolérance d'erreur infinitésimale.

16.4 Représentation des chaînes de caractères

– Les chaînes de caractères obéissent à une logique de représentation totalement différente. Elles nécessitent une table de correspondance liant un symbole à une séquence de bits.
– L'encodage historique ASCII associe 128 caractères à des codes sur 7 bits (stockés sur un octet). Pour pallier ses limites et intégrer l'intégralité des symboles internationaux, la norme moderne UTF-8 est utilisée, encodant les caractères sur une longueur variable d'octets.

Troisième partie

Graphes

Chapitre 17

Représentation native des graphes

17.1 Liste de listes (liste d'adjacence)

- La *liste d'adjacence* sous forme de tableau unidimensionnel est employée lorsque les sommets du graphe sont identifiés par des entiers consécutifs de 0 à $n-1$.
- L'élément d'indice `i` du tableau contient la liste de tous les sommets accessibles directement depuis le sommet `i`.
- La structure diffère selon la nature de la relation :
  - Pour un graphe *non orienté*, chaque arête induit une symétrie dans le stockage : si j appartient à la liste d'indice i, alors i doit obligatoirement figurer dans la liste d'indice j.
  - Pour un graphe *orienté*, l'élément d'indice `i` regroupe exclusivement les successeurs de ce sommet. L'arc inverse n'est pas implicite.
- Dans le cas d'un graphe *pondéré*, chaque voisin n'est plus représenté par un simple entier, mais par un tuple associant le sommet cible et la pondération (le poids) de la liaison.

```
# representation d'un graphe a 3 sommets (0, 1 et 2)

# graphe non oriente : aretes (0, 1) et (1, 2)
liste_non_orientee = [
    [1],       # voisins du sommet 0
    [0, 2],    # voisins du sommet 1
    [1]        # voisins du sommet 2
]

# graphe oriente : arcs 0 -> 1 et 1 -> 2
liste_orientee = [
    [1],       # successeurs de 0
    [2],       # successeurs de 1
    []         # successeurs de 2
]

# graphe oriente et pondere : (successeur, poids)
liste_ponderee = [
    [(1, 4.5)], # l'arc 0 -> 1 a un poids de 4.5
    [(2, 1.2)],
    []
]
```

17.2 Dictionnaire d'adjacence

- L'utilisation d'une simple liste devient caduque lorsque les identifiants des sommets ne sont pas des entiers séquentiels (par exemple, des chaînes de caractères ou des coordonnées). Le *dictionnaire* Python apporte alors la souplesse requise.

– Les clés du dictionnaire modélisent les identifiants des sommets. La valeur associée à une clé est la liste de ses voisins ou de ses successeurs.

– Les logiques de symétrie (graphe non orienté) et d'asymétrie (graphe orienté) y sont strictement identiques à celles de la liste de listes.

– Pour y intégrer une pondération, il est possible d'utiliser une liste de tuples, ou de manière plus idiomatique, un sous-dictionnaire liant chaque voisin à son poids.

```
# representation d'un graphe dont les sommets sont des lettres

dict_non_oriente = {
    "A": ["B"],
    "B": ["A", "C"],
    "C": ["B"]
}

# graphe oriente avec ajout dynamique d'un sommet
dict_oriente = {
    "A": ["B"],
    "B": ["C"],
    "C": []
}
dict_oriente["D"] = ["A"] # ajout de l'arc D -> A

# graphe oriente et pondere avec sous-dictionnaires
dict_pondere = {
    "A": {"B": 4.5},
    "B": {"C": 1.2},
    "C": {}
}
# acces au poids de l'arc A -> B
poids = dict_pondere["A"]["B"]
```

17.3 Matrice d'adjacence

– La *matrice d'adjacence* modélise un graphe d'ordre n via un tableau bidimensionnel de dimension $n \times n$.

– Le coefficient situé à l'indice de ligne `i` et de colonne `j` indique la nature de la liaison allant du sommet i vers le sommet j.

– Pour un graphe non orienté, la matrice est nécessairement symétrique par rapport à sa diagonale principale. Pour un graphe orienté, elle ne l'est généralement pas.

– La modélisation de cette matrice est relative et dépend directement de la convention choisie :
  • Pour un graphe simple, la présence d'une liaison peut être encodée par l'entier 1 et son absence par 0. Les booléens `True` et `False` constituent une alternative sémantique parfaitement valide.
  • Pour un graphe pondéré, le coefficient contient directement la valeur du poids. L'absence de liaison doit alors être traduite par une valeur conventionnelle hors du domaine des poids possibles : souvent la valeur `None`, ou l'infini mathématique (`float('inf')`) pour éviter de fausser les calculs de distances minimales.

```
# graphe non oriente simple : convention avec 1 et 0
matrice_symetrique = [
    [0, 1, 0],
    [1, 0, 1],
    [0, 1, 0]
]

# graphe oriente simple : convention avec True et False
matrice_booleenne = [
    [False, True, False],
    [False, False, True],
```

```
    [False, False, False]
]

# graphe oriente pondere : convention avec l'infini
inf = float('inf')
matrice_ponderee = [
    [inf, 4.5, inf],
    [inf, inf, 1.2],
    [inf, inf, inf]
]
```

– Cette structure permet de vérifier l'existence ou le poids d'une liaison en temps constant ($O(1)$). Cependant, son empreinte mémoire en $O(n^2)$ la rend inadaptée pour les graphes possédant de nombreux sommets mais peu d'arêtes.

17.4 Algorithmes de conversion et d'exploration

– La transformation d'une représentation vers une autre constitue un traitement algorithmique classique. Le passage d'une matrice d'adjacence vers un dictionnaire nécessite d'itérer sur l'intégralité du tableau bidimensionnel au moyen de boucles imbriquées.

```
def matrice_en_dictionnaire(matrice, sommets):
    d = dict()
    n = len(sommets)
    for i in range(n):
        d[sommets[i]] = [] # initialisation de la cle
        for j in range(n):
            if matrice[i][j] == 1:
                d[sommets[i]].append(sommets[j])
    return d
```

– La conversion inverse (d'un dictionnaire vers une matrice) exige l'allocation préalable d'un tableau carré rempli de zéros, de dimension égale au nombre de clés. Il faut ensuite parcourir le dictionnaire pour modifier les valeurs aux indices correspondants.

```
def dictionnaire_en_matrice(d):
    n = len(d)
    sommets = list(d.keys())
    L = [[0] * n for _ in range(n)]

    for i in range(n):
        for j in range(n):
            if sommets[j] in d[sommets[i]]:
                L[i][j] = 1
    return L, sommets
```

– L'exploration d'un graphe implique fréquemment la recherche d'extrema, comme l'identification du sommet possédant le plus grand nombre de connexions. L'algorithme standard de recherche de maximum s'y applique directement.

```
def sommet_degre_max(d):
    sommet_max = None
    degre_max = -1

    for sommet, voisins in d.items():
        degre = len(voisins)
        if degre > degre_max:
            degre_max = degre
            sommet_max = sommet

    return sommet_max
```

– La manipulation matricielle requiert des fonctions utilitaires spécifiques. Il est par exemple fréquent de devoir vérifier si un graphe est non orienté en s'assurant de la symétrie de sa matrice. Pour des raisons d'optimisation, seule la moitié inférieure de la matrice est parcourue :

```
def verif_symetrie(matrice):
    for i in range(len(matrice)):
        for j in range(i): # parcours sous la diagonale
            if matrice[i][j] != matrice[j][i]:
                return False
    return True
```

– De même, la matrice manipulant implicitement des indices entiers, il est nécessaire de créer des outils pour faire le lien avec les étiquettes réelles des sommets (e.g. "A", "B") et pour extraire les voisins d'un sommet donné, en excluant éventuellement ceux déjà visités lors d'un parcours.

```
def determination_numero(etiquettes, nom_noeud):
    # renvoie l'indice correspondant à l'étiquette (ou -1 si introuvable)
    for i in range(len(etiquettes)):
        if etiquettes[i] == nom_noeud:
            return i
    return -1
```

```
def recherche_voisins(mat_adj, noeuds_visites, noeud):
    # extrait la liste des indices des voisins non encore visités
    voisins = []
    for i in range(len(mat_adj)):
        # présence d'une liaison et sommet non présent dans la liste des visités
        if mat_adj[noeud][i] != 0 and i not in noeuds_visites:
            voisins.append(i)
    return voisins
```

Chapitre 18

Parcours de graphes

18.1 Notions

- Parcourir un graphe consiste à visiter systématiquement ses sommets en suivant les arêtes disponibles, à partir d'un sommet initial donné.
- Ces algorithmes permettent de résoudre des problèmes fondamentaux : tester l'existence d'un chemin entre deux sommets, déterminer la connexité, détecter la présence de cycles, ou extraire le plus court chemin.
- L'obstacle majeur réside dans la présence potentielle de cycles (chemins tournant en boucle). Il est par conséquent indispensable d'utiliser une structure de mémorisation, comme un dictionnaire ou une liste, pour marquer les sommets déjà explorés et éviter une exécution infinie.

18.2 Parcours en largeur (BFS)

- Le parcours en largeur (*Breadth-First Search*) explore le graphe en rayonnant : il visite d'abord tous les voisins immédiats du sommet de départ (distance 1), puis les voisins des voisins (distance 2), et ainsi de suite.
- L'implémentation de cette stratégie concentrique repose sur l'utilisation d'une file (structure FIFO).
- Une des implémentations classiques prend en entrée un graphe sous forme de dictionnaire d'adjacence et un sommet de départ, puis marque les sommets visités au fur et à mesure de l'exploration.

```
from collections import deque

def parcours_largeur(G, sommet_depart):
    visited = {s: False for s in G}
    file = deque()

    file.append(sommet_depart)
    visited[sommet_depart] = True

    while file:
        u = file.popleft()

        for v in G[u]:
            if not visited[v]:
                visited[v] = True
                file.append(v)

    return visited
```

- Lors d'un parcours, il est souvent utile de mémoriser l'origine de la découverte de chaque sommet. Le tableau des visites est alors remplacé par un dictionnaire de provenance associant à chaque sommet son sommet parent.

– Le parcours en largeur garantissant l'exploration par le nombre minimum d'étapes, cette approche est idéale pour extraire le plus court chemin dans un graphe non pondéré.

```python
def parcours_largeur_provenance(G, depart):
    provenance = {depart: None}
    file = deque()
    file.append(depart)

    while file:
        u = file.popleft()
        for v in G[u]:
            if v not in provenance:
                provenance[v] = u
                file.append(v)

    return provenance
```

– Dans cet exemple, la restitution du chemin s'effectue en remontant le dictionnaire de provenance, de l'arrivée vers le départ. La distance correspond alors à la longueur du chemin reconstitué, diminuée d'une unité.

```python
def restitution_parcours(provenance, arrivee):
    if arrivee not in provenance:
        return []

    chemin = []
    courant = arrivee

    while courant is not None:
        chemin.append(courant)
        courant = provenance[courant]

    return chemin[::-1] # ordre chronologique
```

– Afin d'illustrer une approche alternative très complète, l'implémentation suivante réalise un parcours en largeur sur un graphe défini par une matrice d'adjacence, et substitue l'objet `deque` par une liste Python classique. Cette fonction intègre le calcul de la provenance de chaque sommet, sous forme de liste cette fois, ainsi que la construction de la matrice de l'arbre de parcours. Elle exploite les fonctions utilitaires `determination_numero` et `recherche_voisins` préalablement définies.

```python
def parcours_largeur_graphe(mat_adj, depart, etiquettes):
    n = len(mat_adj)
    num_noeud_depart = determination_numero(etiquettes, depart)

    file = [num_noeud_depart]
    noeuds_visites = [num_noeud_depart]

    # initialisation du tableau de provenance et de la matrice de l'arbre
    provenance = [0] * n
    provenance[num_noeud_depart] = num_noeud_depart # selon la convention
    mat_arbre = [[0] * n for i in range(n)]

    while len(file) != 0:
        noeud_courant = file[0]
        file[0:1] = []

        for v in recherche_voisins(mat_adj, noeuds_visites, noeud_courant):
            noeuds_visites.append(v)
            file.append(v)

            # mise a jour de la provenance et construction de l'arbre
            provenance[v] = noeud_courant
            mat_arbre[noeud_courant][v], mat_arbre[v][noeud_courant] = 1, 1

    # traduction des indices en etiquettes lisibles
```

```
    etiquettes_visites = [etiquettes[i] for i in noeuds_visites]

    return etiquettes_visites, mat_arbre, provenance
```

– L'évaluation de la connexité du graphe se déduit directement de ce parcours.

```
liste_noeuds, mat_arbre, provenance = parcours_largeur_graphe(mat_adj, depart,
    etiquettes)

if len(liste_noeuds) == len(mat_adj):
    print("le graphe est connexe")
else:
    print("le graphe n'est pas connexe")
```

– À l'aide du tableau de `provenance` généré, il est possible de restituer le chemin le plus court sous forme d'une chaîne de caractères reliant le sommet de départ à celui d'arrivée. L'algorithme remonte les indices de provenance successifs, tout en traduisant ces indices grâce à la liste des `etiquettes`.

```
def restitution_parcours(provenance, etiquettes, depart, arrivee):
    s = ""
    num_depart = determination_numero(etiquettes, depart)
    num_arrivee = determination_numero(etiquettes, arrivee)

    num = num_arrivee
    while num != num_depart:
        s = " " + etiquettes[num] + s
        num = provenance[num]

    return etiquettes[num_depart] + s
```

18.3 Parcours en profondeur (DFS)

– Le parcours en profondeur (*Depth-First Search*) explore le graphe en s'enfonçant le plus loin possible dans une ramification avant de revenir sur ses pas à la rencontre d'un cul-de-sac.

– L'implémentation itérative repose sur l'utilisation d'une pile (structure LIFO). Un sommet n'est marqué comme visité qu'au moment de son extraction de la pile.

```
from collections import deque

def parcours_profondeur_iteratif(G, depart):
    visites = []
    pile = deque()
    pile.append(depart)

    while pile:
        u = pile.pop()
        if u not in visites:
            visites.append(u)
            for v in G[u]:
                if v not in visites:
                    pile.append(v)

    return visites
```

– Le mécanisme d'empilement correspondant exactement au fonctionnement de la pile d'exécution interne du langage, ce parcours s'écrit de manière élégante via la récursivité.

```
def parcours_profondeur_recursif(G, u, visites=None):
    if visites is None:
        visites = []

    visites.append(u)

```

```
    for v in G[u]:
        if v not in visites:
            parcours_profondeur_recursif(G, v, visites)

    return visites
```

– Une application directe de ce parcours est l'évaluation de la connexité. Un graphe non orienté est connexe s'il est possible d'atteindre l'intégralité de ses nœuds depuis n'importe quel point. Il suffit d'initialiser le parcours depuis un sommet arbitraire et de vérifier que tous les sommets ont été atteints.

```
def est_connexe(G):
    if len(G) == 0:
        return True

    depart = list(G.keys())[0]
    visites = parcours_profondeur_recursif(G, depart)

    return len(visites) == len(G)
```

– Le parcours en profondeur constitue également la méthode de référence pour détecter la présence d'un cycle dans un graphe non orienté.
Le principe requiert de transmettre le sommet parent lors de l'exploration. Si l'algorithme rencontre un sommet voisin déjà visité, et que ce voisin n'est pas le parent direct dont on provient, l'existence d'une boucle fermée est avérée.

```
def contient_cycle(G, u, parent=None, visites=None):
    if visites is None:
        visites = []

    visites.append(u)

    for v in G[u]:
        if v not in visites:
            if contient_cycle(G, v, u, visites):
                return True
        elif v != parent:
            return True

    return False
```

– De façon analogue, le parcours en profondeur itératif peut être adapté pour traiter une matrice d'adjacence, tout en construisant simultanément l'arbre de parcours et le tableau des provenances. L'emploi d'une liste Python classique pour modéliser la pile y est naturellement optimisé.

```
def parcours_profondeur_graphe(mat_adj, depart, etiquettes):
    n = len(mat_adj)
    num_noeud_depart = determination_numero(etiquettes, depart)

    pile = [num_noeud_depart]
    noeuds_visites = [num_noeud_depart]

    provenance = [0] * n
    provenance[num_noeud_depart] = num_noeud_depart
    mat_arbre = [[0] * n for i in range(n)]

    while len(pile) != 0:
        noeud_courant = pile.pop()

        for v in recherche_voisins(mat_adj, noeuds_visites, noeud_courant):
            noeuds_visites.append(v)
            pile.append(v)

            provenance[v] = noeud_courant
            mat_arbre[noeud_courant][v], mat_arbre[v][noeud_courant] = 1, 1
```

```
21
22      etiquettes_visites = [etiquettes[i] for i in noeuds_visites]
23
24      return etiquettes_visites, mat_arbre, provenance
```

Quatrième partie

Bases de données (SQL)

Chapitre 19

Bases de données

19.1 Architecture

- Une *base de données* constitue un ensemble structuré d'informations. Son interrogation s'effectue généralement via un *SGBD* (Système de Gestion de Bases de Données) exploitant le langage standardisé SQL.
- L'architecture matérielle se décline selon le volume et le partage des données :
  - *Architecture à 1 niveau* : fonctionnement local, sans partage.
  - *Architecture à 2 niveaux* (client-serveur) : le serveur héberge les données, le client envoie les requêtes et peut gérer une partie du traitement.
  - *Architecture à 3 niveaux* (ou 3 tiers) : sépare le client (affichage), le serveur d'application (traitement logique et requêtes) et le serveur de base de données (stockage).

19.2 Vocabulaire et modélisation

- Les données sont organisées sous forme de *tables* (ou *relations*).
- Une ligne de la table est appelée un *enregistrement* (ou *tuple*, ou *vecteur*). Elle représente un objet unique.
- Une colonne est appelée un *attribut*.
- L'ensemble des valeurs autorisées pour un attribut constitue son *domaine*. L'ensemble des attributs définit le *schéma* de la table.
- L'identification unique d'un enregistrement est garantie par la *clé primaire* (souvent un identifiant abstrait de type entier, noté `id`).
- La *clé étrangère* est un attribut faisant référence à la clé primaire d'une autre table, permettant de lier les données entre elles.
- La modélisation des liens (*cardinalités*) détermine la structure :
  - *Relation 1 à 1* : correspondance bijective (rare).
  - *Relation 1 à plusieurs* : modélisée par l'ajout d'une clé étrangère dans la table portant la multiplicité.
  - *Relation plusieurs à plusieurs* : impose la création d'une *table d'association* (ou de liaison) contenant au minimum deux clés étrangères référençant les tables concernées.

19.3 Extraction et restriction

- Toute instruction SQL s'achève par un point-virgule.
- La *projection* consiste à sélectionner des colonnes spécifiques au moyen de la clause `SELECT`. Le caractère `*` permet de sélectionner l'intégralité des attributs.

– La notation pointée `table.attribut` est requise pour lever toute ambiguïté lors de la manipulation de multiples tables.

– La *restriction* filtre les lignes vérifiant une condition booléenne via la clause `WHERE`.

– Les opérateurs de comparaison classiques sont `=`, `<>` (différent de), `>`, `<`, `>=`, `<=`. Ils se combinent avec les opérateurs logiques `AND`, `OR`, `NOT`.

```
SELECT titre, artiste
FROM chanson
WHERE duree > 180 AND artiste = 'Agar Agar';
```

– L'opérateur `LIKE` permet la recherche de motifs dans les chaînes de caractères. Le caractère `_` se substitue à un unique caractère exact, tandis que `%` (placé au début, au milieu ou à la fin) remplace une séquence de zéro, un ou plusieurs caractères.

```
SELECT artiste FROM chanson WHERE nom LIKE '%life%';
```

19.4 Tris, limites et opérations ensemblistes

– Le tri des résultats s'opère avec `ORDER BY`, suivi de `ASC` (croissant, par défaut) ou `DESC` (décroissant). L'ordre s'applique également de manière lexicographique sur les chaînes de caractères.

– L'élimination des doublons dans le résultat d'une projection s'effectue en plaçant le mot-clé `DISTINCT` immédiatement après `SELECT`.

– La limitation du nombre de résultats renvoyés utilise `LIMIT`. Le décalage de l'indice de départ s'obtient avec `OFFSET` (l'indice initial étant 0).

```
SELECT DISTINCT artiste
FROM chanson
ORDER BY artiste ASC
LIMIT 4 OFFSET 1; -- du deuxième au cinquième
```

– Les opérations ensemblistes exigent que les tables manipulées aient des schémas compatibles (même nombre d'attributs et types compatibles). Elles incluent :
  • `UNION` : fusionne les lignes des deux tables.
  • `INTERSECT` : conserve les lignes communes.
  • `EXCEPT` : conserve les lignes de la première table absentes de la seconde.

```
SELECT * FROM playlist_A
UNION
SELECT * FROM playlist_B;
```

19.5 Croisement de données et jointures

– Le *produit cartésien* génère toutes les combinaisons possibles entre les lignes de plusieurs tables. Il s'obtient en listant les tables séparées par des virgules dans la clause `FROM`.

```
SELECT * FROM chanson, genre_musical;
```

– La *jointure* recolle deux tables selon un critère logique (généralement l'égalité entre une clé étrangère et une clé primaire). Elle s'écrit formellement avec `JOIN ... ON ...`. Une alternative syntaxique consiste à réaliser un produit cartésien suivi d'une restriction `WHERE`.

```
SELECT *
FROM chanson
JOIN genre_musical ON chanson.id_genre = genre_musical.id;
```

– L'*autojointure* permet de joindre une table avec elle-même. Il est alors rigoureusement indispensable d'utiliser des *alias* (mot-clé `AS`) pour distinguer les deux instances de la table.

```sql
SELECT a.titre, b.titre
FROM chanson AS a, chanson AS b
WHERE a.duree + b.duree <= 540;
```

19.6 Agrégation et calculs

– Le langage SQL intègre des fonctions de calcul s'appliquant sur un ensemble de valeurs : `COUNT()` (dénombrement), `SUM()` (somme), `AVG()` (moyenne), `MAX()` et `MIN()`.

```sql
SELECT COUNT(DISTINCT artiste) FROM chanson;
SELECT AVG(duree) FROM chanson;
SELECT nom, MAX(duree) FROM chanson;
```

– La clause `GROUP BY` partitionne la table en sous-groupes (agrégats) en fonction des valeurs d'un attribut. La fonction d'agrégation spécifiée dans le `SELECT` s'applique alors indépendamment sur chaque sous-groupe.

– Lors d'une agrégation, le `SELECT` ne peut contenir *que* l'attribut de partitionnement et des fonctions d'agrégation.

– La clause `HAVING` applique une condition de restriction *après* l'opération de regroupement et de calcul, contrairement au `WHERE` qui filtre les lignes *avant* l'agrégation.

```sql
-- Compte, pour chaque artiste, le nombre de chansons synthwave, et ne garde que
    ceux qui en ont au moins 2.
SELECT artiste, COUNT(*)
FROM chanson
WHERE genre = 'synthwave'
GROUP BY artiste
HAVING COUNT(*) >= 2;
```

Cinquième partie

Algorithmes pour l'intelligence artificielle

Sixième partie

Algorithmique numérique

Chapitre 20

Intégration numérique

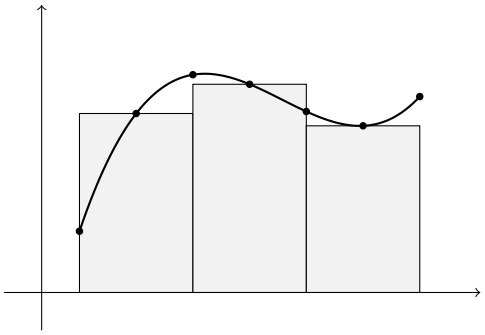
20.1 Introduction

- L'intégration numérique a pour objectif d'évaluer de façon approchée l'intégrale d'une fonction sur un intervalle borné $[a, b]$. Cette évaluation s'effectue à partir d'un ensemble fini de points, issus d'un calcul analytique ou de mesures physiques.
- La répartition des points sur l'axe des abscisses est généralement uniforme, définie par un pas d'échantillonnage constant noté h.
- Le principe fondamental repose sur l'interpolation polynomiale par morceaux. L'intervalle est découpé en sous-intervalles sur lesquels la fonction est approchée par un polynôme de degré faible (généralement 0, 1 ou 2), dont l'intégrale est aisée à calculer.
- La précision du calcul s'améliore en augmentant le nombre de points n (ce qui diminue le pas h) ou en élevant le degré du polynôme d'interpolation, sous réserve que la fonction étudiée présente de bonnes propriétés de continuité.

20.2 Méthode du point milieu

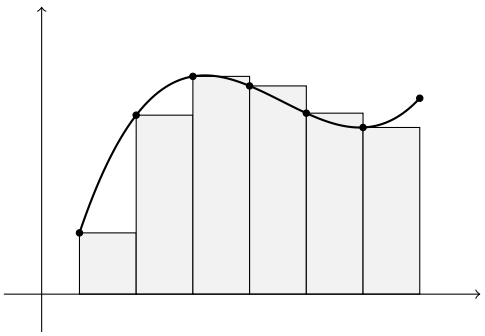
- La méthode du point milieu s'appuie sur une interpolation polynomiale de degré 0. La fonction est considérée comme constante sur chaque intervalle.
- Dans son approche théorique centrée sur des intervalles de largeur $2h$, la valeur de la fonction est évaluée au milieu de l'intervalle. L'intégrale s'écrit alors (pour un nombre de points n impair)

$$I = 2h \sum_{i=1}^{(n-1)/2} y_{2i}.$$



- Cette méthode ne requière en réalité qu'un échantillonnage de $2h$. L'intégration s'effectue alors de manière causale en utilisant la valeur du point courant sur une largeur h :

$$I = h \sum_{i=1}^{n-1} y_i.$$



Cette seconde méthode, bien que plus directe à implémenter, induit un décalage temporel systématique. L'aire calculée sur l'intervalle $[x_i, x_{i+1}]$ utilise la valeur y_i. Le résultat numérique est donc une approximation de l'intégrale d'une fonction déphasée, ce qui peut constituer une source d'erreur significative dans certaines applications.

– Afin d'implémenter cette méthode algorithmiquement, on propose la fonction suivante qui prend en argument une liste `Y` contenant les ordonnées successives des points, ainsi que le pas d'échantillonnage `h`.

```
1  def integ_point_gauche(Y, h):
2      s = 0
3      for i in range(len(Y) - 1): # sans le dernier point
4          s += h * Y[i]
5      return s
```

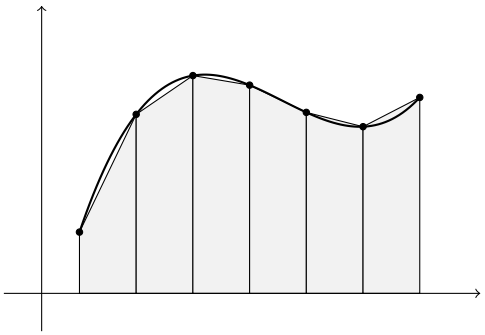
– Notons qu'on peut aussi employer le « point de droite ».

20.3 Méthode des trapèzes

– La méthode des trapèzes affine l'approximation en utilisant une interpolation linéaire (polynôme de degré 1) entre chaque couple de points consécutifs. L'aire sous la courbe est ainsi assimilée à une somme d'aires de trapèzes.

– En sommant les aires sur l'intervalle global, les ordonnées des points intérieurs sont comptabilisées deux fois, tandis que celles des extrémités ne le sont qu'une seule fois. La formule générale s'écrit donc

$$I = h \sum_{i=}^{n-1} \frac{y_i + y_{i+1}}{2}.$$



– Une implémentation de cette méthode en Python est la suivante.

```
1  def integrer_trapezes(Y, h):
2      s = 0
3      for i in range(len(Y) - 1):
4          s += (Y[i] + Y[i + 1]) / 2
5      return h * s
```

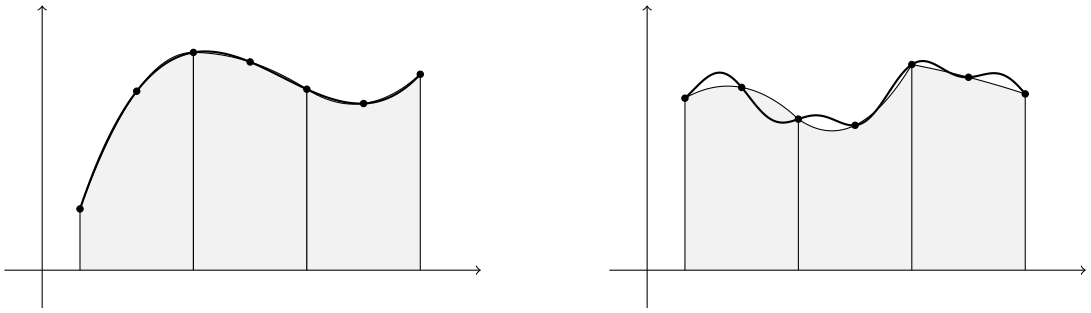
20.4 Méthode de Simpson

- La méthode de Simpson pousse la précision à un niveau supérieur en s'appuyant sur une interpolation quadratique, c'est-à-dire en approximant la fonction par un arc de parabole (polynôme de degré 2).
- Une parabole étant définie de manière unique par trois points, le calcul s'effectue par segments successifs de largeur $2h$, chacun délimité par un triplet de points $(y_{2i-1}, y_{2i}, y_{2i+1})$. Cette contrainte impose que le nombre total de points n soit impair.
- L'intégration analytique de la parabole unique passant par ces trois points aboutit à la formule locale, connue sous le nom de formule de Simpson

$$I_{local} = \frac{h}{3}(y_{2i-1} + 4y_{2i} + y_{2i+1}).$$

- En appliquant cette formule à chaque segment de trois points sur l'ensemble de l'intervalle, la valeur approchée globale de l'intégrale est donnée par la somme suivante :

$$I = \frac{h}{3} \sum_{i=1}^{(n-1)/2} (y_{2i-1} + 4y_{2i} + y_{2i+1}).$$



- Bien que mathématiquement très précise pour les fonctions lisses, cette méthode est plus sensible aux bruits de mesure. Et dans un contexte temps réel, elle requiert la connaissance de trois points pour effectuer un calcul, induisant un retard temporel de $2h$.
- L'implémentation de cet algorithme parcourt la liste des ordonnées par pas de deux afin de traiter les points par triplets successifs.

```
1  def integ_simpson(Y, h): # le nombre de points doit etre impair
2      s = 0
3      for i in range(0, len(Y) - 2, 2): # pas de 2
4          s += Y[i] + 4 * Y[i+1] + Y[i+2]
5
6      return s * (h / 3)
```